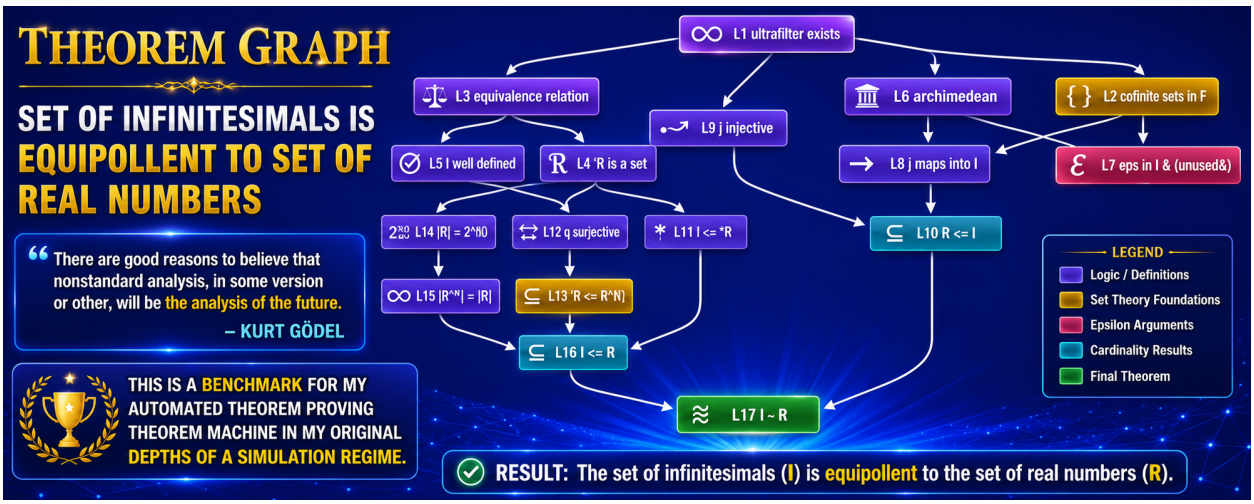
Predator 7.1 Documentation and Cross-Branch Reuse Rate Theory

with efforts toward lower bounds on the settlement length of P vs NP

Brian Tenneson

July 2026
Revision 2



Abstract

This document documents a proof search over Metamath's `set.mm`, develops a theory of *cross-branch reuse* in dovetailed proof search, and asks what can honestly be said about the length of a shortest proof settling P vs NP.

Part I documents Predator_7.1 and its certificate verifier (CV). All 47,572 `set.mm` proofs verify, and the four certificates Predator_7.1 has emitted verify in full corpus context. It then reports a defect in those four results: each is a one-step citation of a statement already present in the database under a different label. The infrastructure is correct; the mathematical content of the four proofs is zero. A corrected evaluation protocol is given, together with the measurement that makes the defect a corpus-wide hazard rather than an accident — 16.8% of `set.mm`'s logical assertions share their statement with at least one other label.

Part II introduces the *settlement machine* $M^{\otimes}$, which pursues a conjecture c and its negation on alternating stages over a shared lemma store, and the *cross-branch reuse rate* κ. Three theorems are proved. Theorem 9.1 shows κ is a property of the search policy and not of the theory. Theorem 10.1 identifies cross-branch insertion with the derived-rule construction of the Conservative Compilation Theorem, so that the branch pursuing the false side of a conjecture lowers the transaction count of the branch that will halt. Theorem 11.2 reduces a policy-relative speedup to a combinatorial condition on chains, and identifies precisely the additional hypothesis

— a proof-length lower bound — needed to upgrade it to the Abstract Asymptotic Speedup Principle.

Part III observes that the CV is a polynomial-time verifier in the sense of the definition of NP, making the enterprise reflexive. Four routes to a lower bound on ‖P ≠ NP‖ are examined. Three fail, for reasons worth recording. The fourth, the *definitional floor*, gives a soft but computable bound resting on a checkable fact: `set.mm` contains no machine model, no time bound, and no complexity class.

The unifying observation is stated in Section 16. The obstacle to completing Part II and the obstacle to completing Part III are the same obstacle — both require lower bounds on proof length in a strong system, and no technique for obtaining those is currently known.

Contents

1 Introduction **4**
- 1.1 The question this document is about . . . 4
- 1.2 Contributions . . . 4
- 1.3 Relation to *Depths of a Simulation* . . . 4
- 1.4 What changed in Revision 2 . . . 5

2 Note on sources and on what is measured **6**

Part I — Predator_7.1 **7**

3 Architecture **7**
- 3.1 The two-phase split . . . 7
- 3.2 What changed from 7.0: unification . . . 7
- 3.3 Search control . . . 8

4 The verification result **8**
- 4.1 The verifier . . . 8
- 4.2 The certificate container, and the number 47,573 . . . 9

5 The defect in the four results **9**
- 5.1 What the four proofs actually are . . . 9
- 5.2 Why the existing safeguard does not catch this . . . 10
- 5.3 How large the hazard is . . . 10
- 5.4 A corrected protocol . . . 10

Part II — Cross-branch reuse rate theory **11**

6 The preorder on SICs, and which one is meant **11**
- 6.1 Weak simulation does collapse . . . 11
- 6.2 Clocked simulation does not . . . 11

7 Quantities **12**

8 The settlement machine **13**

9 First result: reuse is a property of the policy **14**

10 Second result: cross-branch reuse is online compilation **15**

11 Third result: chain deficits and speedup **16**
11.1 What the speedup principle actually requires 16
11.2 What a chain looks like, and what a layer looks like 17

12 Measurement protocol **18**

Part III — Lower bounds on P vs NP **19**

13 The certificate verifier is an NP verifier **19**

14 Four routes to a lower bound **20**
14.1 Route 1: counting. Fails, and instructively 20
14.2 Route 2: the proof horizon. Real, but bounds the machine 20
14.3 Route 3: the barriers. Bound the content, not the length 20
14.4 Route 4: the definitional floor 21

15 What is actually known about proof length lower bounds **22**
15.1 The unit problem in $\|\cdot\|$ 22

16 Conclusion **24**
16.1 What is proved 24
16.2 What is measured 24
16.3 What is conjectured, and what is merely hoped 24
16.4 The unifying observation 24
16.5 Next steps, in order of tractability 25

17 A Collection of Objects Discussed Herein **26**

Appendices **30**

A History of Predator **30**

B The four certificates in full **31**

C Source code **32**
C.1 `metamath_cv.py` — the certificate verifier 32
C.2 `setmm_grammar.py` — grammar extraction and parsing 42
C.3 `predator71.py` — the prover 48
C.4 `tau.py` — transaction and materialisation counts 56

1 Introduction

1.1 The question this document is about

An automated theorem prover that searches for a proof of c and fails tells you almost nothing. It may have failed because c is false, because c is independent, because the proof is longer than the budget, or because the policy looked in the wrong place. A prover that searches for c and $\neg c$ at once — a *dovetailed* or *settlement* search — at least distinguishes the first case from the others when the refutation is reachable.

That much is classical. The question taken up here is quantitative and, so far as I have been able to determine, unasked: when a settlement search runs both branches over a shared store of proved lemmas, **how often does a lemma proved while pursuing c get used while pursuing $\neg c$?** Call that the cross-branch reuse rate κ.

The question matters because the answer decides whether dovetailing is a bargain or a tax. If $\kappa = 0$ the two searches are independent, dovetailing costs a factor of two, and it buys only the ability to detect refutation. If $\kappa > 0$ the branch that will never halt is nonetheless manufacturing lemmas for the branch that will, and — by Theorem 10.1 below — doing so is formally identical to compiling derived rules, which the Conservative Compilation Theorem of *Depths of a Simulation* shows lowers transaction counts.

1.2 Contributions

(1) **A working certificate verifier and prover**, documented in Part I, with the trust asymmetry made explicit: 916 lines are trusted, the rest is not.

(2) **A negative result about the prover's demonstrated capability** (Section 5), including the corpus measurement that generalises it and a corrected evaluation protocol.

(3) **The settlement machine and the reuse rate** (Definitions 8.1–8.6), with the transaction count of *Depths of a Simulation* extended to two-sided targets as the *settlement count* σ.

(4) **Three theorems** on κ, each tied to a specific result of that paper.

(5) **The definitional floor** (Definition 14.2), a lower bound on proof length that is soft but actually computable, in a setting where every sharp bound is out of reach.

(6) **A unification** (Section 16): the open problem in Part II and the open problem in Part III have the same shape.

1.3 Relation to *Depths of a Simulation*

References of the form `thm:compilation` are to labels in *Depths of a Simulation*, version 45, hereafter **v45**. This document is a companion, not a summary: it uses v45's definitions unchanged wherever they exist, and introduces new ones only where a needed object is absent. Section 17 records, for every object used, whether it originates in v45 or here.

Two conventions of v45 are load-bearing throughout and are worth stating at the outset.

- $\|\varphi\|_F(\Gamma)$ is the **line count** of a shortest proof, $\min\{|\pi|\}$. The size-relativised variant $\|\varphi\|_F^{\text{size}}(\Gamma)$, summing formula lengths, is also defined in v45, which notes that the entire apparatus relativises under $|\pi| \mapsto \text{size}(\pi)$.

- $\preceq$ without adornment means **clocked** deterministic simulation. This matters more than it looks; see Section 6.

1.4 What changed in Revision 2

Revision 1 of this document contained errors, all of which were found by checking its claims against the text of v45 rather than against my memory of it. They are listed here rather than silently repaired, because two of them were load-bearing.

(a) **The preorder section was wrong.** Revision 1 asserted that every SIC simulates every other, that the quotient poset is a single point, and that the settlement-region programme is therefore dead. That is true of *weak* simulation only. v45's principal relation is *clocked* simulation, which provably does not collapse. Section 6 is rewritten, and the conclusion is reversed.

(b) **The Proof-Horizon Theorem was attributed to the wrong machine.** Revision 1 wrote $\tau = \|\varphi\|$ for the canonical rule-enumeration SIC $E_{F,Y}$. But $E_{F,Y}$ has $H \equiv 0$ and never halts. The identity holds for the canonical breadth-first target-search SIC $B_{F,Y,\varphi}$. Corrected in Section 7.

(c) **A qualifier was dropped.** Revision 1 wrote that no target-search SIC reaches φ before $\|\varphi\|$. v45 says no target-search SIC *based solely on shortest proof length* does. Restored.

(d) **Theorem 11.2 did not witness what it claimed.** The Abstract Asymptotic Speedup Principle requires a lower bound on *every* proof in the original system — a proof-complexity statement. Revision 1's hypothesis was about a *policy*. The theorem is restated as a policy-relative speedup, and Remark 11.3 states exactly what must be added.

(e) **The settlement machine was not a target-search ATP.** v45's definition requires that halting imply the target is present. A machine that halts on $\neg c$ violates it. Fixed by Definition 8.4, which introduces the settlement count σ, and Proposition 8.2, which verifies the SIC axioms.

(f) **The reuse rate was claimed to equal 1 where it is merely degenerate.** Corrected in Theorem 9.1, which now distinguishes availability from use.

(g) **The "unit problem" overstated its novelty.** v45 already has a size-relativised measure. The correct claim concerns a third measure, logical-line count, which it does not have. Restated in Section 15.1.

(h) **Notation collision.** Revision 1 used ρ for the reuse rate; v45 uses ρ for an admissible clock. The reuse rate is now κ.

(i) **Three numerical errors**, found by recomputing rather than recalling. The median raw proof length is 173, not 183. The figure 21.4 was quoted as the ratio of medians when it is the ratio of aggregates, whose correct value is 21.2; the ratio of medians is 14.4. And the corpus contains 1,441 syntax *axioms*, the figure 1,445 being the count of syntax *assertions*, which includes four proved ones. All three are corrected in Section 15.1 and the table on page 22.

(j) **An overstated equivalence.** Revision 1 claimed $\Delta(c) > 0 \iff \kappa > 0$. Only one direction holds; see Remark 10.2.

(k) **A mislabelled column.** The "Position" column of the certificate table in Section 5 reported *line numbers in the file*, not ordinal positions among assertions, while the neighbouring dependency table in Part III reported genuine ordinal positions. The two were presented in the same units. Corrected; the gaps are smaller than Revision 1 implied, though the conclusion is unaffected.

Items (a), (d) and (e) were structural: each invalidated a claim rather than a figure. That three of the eight were found only by reading v45's actual text, and three more only by recomputing from `set.mm`, is the argument for the provenance tables of Section 17.

2 Note on sources and on what is measured

Every numerical claim about `set.mm` was computed from the copy in the working folder at the time of writing. Estimates are labelled as estimates in the sentence that makes them. Where something is not known — by me or by anyone — that is stated.

Quantity	Value
Total assertions (`$a` and `$p`)	50,572
Axioms and definitions (`$a`)	3,000
syntax axioms	1,441
logical `$a` (of which `ax-*`: 126; `df-*`: 1,433)	1,559
Proved theorems (`$p`)	47,572
Logical assertions (conclusion begins `|-`)	49,127
Syntax assertions (`$a` and the four syntax `$p`)	1,445
Constants	1,439
Distinct logical statements	43,836
Statements carried by more than one label	2,946
Labels involved in such a duplication	8,237

The last three rows are the subject of Section 5. The count of *true* axioms — the 126 `ax-*` statements — is worth separating from the 3,000 `$a` total, since 1,441 of those are grammar and 1,433 are definitions.

Part I — Predator_7.1

3 Architecture

Predator_7.1 is three programs with a deliberate asymmetry of trust.

Component	File	Role
Certificate verifier	`metamath_cv.py`	trusted, small, judges
Grammar	`setmm_grammar.py`	untrusted, parses
Search	`predator71.py`	untrusted, large, proposes

This is the De Bruijn criterion: an arbitrarily large and arbitrarily clever searcher, and a small verifier that does not trust it. The searcher may be wrong, may be unsound, may be a neural network or a random number generator — nothing it reports is believed. What is believed is the verifier's verdict on the artefact the searcher emits.

No result in this document depends on the correctness of `predator71.py`. It depends on the correctness of `metamath_cv.py`, which is 916 lines, has a self-test, and reproduces the accepted status of all 47,572 `set.mm` proofs.

3.1 The two-phase split

A Metamath proof is a sequence of labels consumed by a stack machine, and the overwhelming majority of those labels are not inference — they are *formula construction*. Measured over a random sample of 6,000 `set.mm` proofs decompressed by the CV, the median raw proof length is 173 steps against a median logical length of 12, and 95.3% of all steps executed are formula-building. See Section 15.1 for why the two ratios these figures support — 14.4 by medians, 21.2 in aggregate — are different numbers and must not be interchanged.

This is exploitable because a parse tree *is* a Metamath syntax proof. A node with rule L and children $k_1, \ldots, k_n$ serialises in reverse Polish as

$$\text{proof}(k_1) \ \cdots \ \text{proof}(k_n) \ L.$$

So the 95% of steps that are notation are *generated* by walking a tree, and only the remaining 5% are searched for. The grammar is not written by hand; it is extracted from the 1,441 syntax axioms already in the database.

3.2 What changed from 7.0: unification

Predator_7.0 searched by *matching*, which is one-way: the goal is ground and the assertion's conclusion carries the variables. That works only for *determined* assertions — those in which every variable occurring in a hypothesis also occurs in the conclusion.

`ax-mp` is not determined. Its conclusion is the bare variable `ps`, so matching it against a goal G binds `ps` $:= G$ and leaves `ph` — the entire antecedent — with no value. `syl` fails identically. These are the two most-cited logical labels in `set.mm`.

Version 7.1 replaces matching with unification over two namespaces: assertion variables `ph`, `ps`, `A`, ..., renamed apart at each use, and metavariables `?1`, `?2`, ..., standing for not-yet-known subterms. Applying `ax-mp` to goal G binds `ps` $:= G$ and turns `ph` into a fresh metavariable `?1`, so the second subgoal is (`?1` $\rightarrow G$). That subgoal is not unconstrained: its consequent is fixed, so only

assertions concluding something of the form $(\_ \to G)$ can unify, and whichever does determines `?1`.

The principle is that a free metavariable is harmless provided some later step determines it. What must never happen is that a step is required to *invent* a formula from nothing.

3.3 Search control

Three controls matter, and each exists because its absence produced a specific failure.

Most-constrained goal first. Expanding the leftmost open goal makes `ax-mp` an infinite regress: it turns G into $(?1 \to G)$ with `?1` free, and `?1` can be attacked by `ax-mp` again. `pick_goal` orders goals by (bare metavariable last, fewest metavariables, largest term).

Indexed hypothesis slots. Once goals are solved out of declaration order, appending completed subproofs in solution order emits them in the wrong order. The verifier caught this as `hypothesis mismatch at ax-mp`. `Step.subs` is a fixed-length list, one slot per `$e` hypothesis, filled by index.

Closers before openers. Assertions with no `$e` hypotheses close a branch outright; those with hypotheses extend it. Ranking a mixed candidate list and truncating it to a fixed width silently discards closers sitting past the cut — which made `prcom` fail after one expansion, since `pm2.01` sat past position 48. The index is split so closers and openers are capped separately.

4 The verification result

4.1 The verifier

`metamath_cv.py` verifies all 47,572 `set.mm` proofs with zero failures in approximately 178 seconds. Two defects found during development are worth recording because both produced *plausible* wrong answers rather than crashes.

The `Z` backreference defect. Compressed proofs mark reusable subproofs with `Z`. The first implementation appended to the saved list on every step rather than only at a `Z`, producing a decoding that was self-consistent and wrong. It was caught only by a cross-encoding test: three encodings of one proof — normal, compressed, compressed with backreferences — must decode to the same step sequence. They gave 34, 34 and 26.

The disjoint-variable defect. A theorem's own proof may introduce *dummy* variables appearing in no hypothesis and no conclusion. Checking the proof against the mandatory disjoint-variable set rather than against every pair active in scope rejects five correct proofs: `equid`, `ax7`, `exgen`, `spnfw`, `spsv`. The fix is `all_dvs`, which collects every disjoint pair active in the frame stack.

Both defects share a shape. Neither made the verifier crash; each made it confidently wrong on a small subset. A verifier that is right about 47,567 proofs and wrong about 5 is not 99.99% correct, it is broken, and the only reason to trust the current one is that the failure modes were sought deliberately rather than waited for.

4.2 The certificate container, and the number 47,573

```
$( Predator_7.1 proof of axin1 $)
$[ set.mm $]
chk $p |- ( ( ph -> -. ph ) -> -. ph ) $= wph pm2.01 $.
```

The `$[ set.mm $]` line splices the entire database in. The verifier reads all of `set.mm` and then one further statement, `chk`. Hence the header `parsed in 9.7s: 3,000 axioms, 47,573 theorems, 1,439 constants`, and hence $47{,}573 = 47{,}572 + 1$. The certificate is a one-statement extension of `set.mm`: it modifies nothing, may cite only what precedes it, and is checked against the whole corpus by a program sharing no code with the search that produced it.

Certificate	Parse (s)	Verify (s)	Result
`axin1_p71.mm`	9.7	196.9	47,573 verified, 0 failed
`falanfal_p71.mm`	9.1	254.2	47,573 verified, 0 failed
`tbw-ax4_p71.mm`	11.7	195.0	47,573 verified, 0 failed
`axia1_p71.mm`	8.9	165.8	47,573 verified, 0 failed

Remark 4.1 (on the cost)**.** Each run re-verifies 47,572 proofs already known to check in order to check one that was not. A `--only` flag restricting verification to labels absent from the included database would reduce each run from roughly 200 seconds to the 10-second parse. The declining throughput within each run — from 11,235 proofs per second at position 2,000 to 238 at position 46,000 — is not a defect but a consequence of `set.mm` being ordered roughly by difficulty.

5 The defect in the four results

5.1 What the four proofs actually are

Target	Position	Cites	Position	Gap
`axin1`	2,720	`pm2.01`	189	2,531
`falanfal`	1,603	`anidm`	573	1,030
`tbw-ax4`	1,730	`falim`	1,584	146
`axia1`	2,717	`simpl`	486	2,231

"Position" here means index among the 50,572 assertions in declaration order, which is the ordering the searcher's cutoff respects. It is not the line number in the file.

Target statement	Cited statement	Relation
`|- ( ( ph -> -. ph ) -> -. ph )`	`|- ( ( ph -> -. ph ) -> -. ph )`	identical
`|- ( ( F. /\ F. ) <-> F. )`	`|- ( ( ph /\ ph ) <-> ph )`	instance
`|- ( F. -> ph )`	`|- ( F. -> ph )`	identical
`|- ( ( ph /\ ps ) -> ph )`	`|- ( ( ph /\ ps ) -> ph )`	identical

Three of the four are verbatim duplicates. The fourth is a one-variable substitution instance. Every proof has logical depth 1: find the earlier label that already says this, cite it.

The proofs are valid and the verifier is right to accept them. But what Predator_7.1 discovered in each case is that *the theorem was already in the database under another name.* That is a lookup, not a proof.

5.2 Why the existing safeguard does not catch this

Predator_7.1 already restricts its index:

```
cut = mm.order.index(a.label)
idx = Index(mm, by_tc, upto=cut, ...)
```

so the searcher sees only assertions declared strictly before the target. It cannot cite the target itself and cannot reach anything downstream. That is the correct and standard protocol, and it is necessary.

It is not sufficient. It excludes the label; it does not exclude the *statement*. `set.mm` re-derives the axioms of several alternative propositional calculi in later sections — which is why `axin1` at position 2,720 says exactly what `pm2.01` said at position 189, some 2,531 assertions earlier. Any ordinal cutoff leaves the duplicate visible.

5.3 How large the hazard is

Logical assertions in `set.mm`	49,127
Distinct logical statements	43,836
Statements carried by > 1 label	2,946
Labels involved in a duplication	8,237
Fraction of logical assertions so involved	**16.8**%

Roughly one in six logical assertions in `set.mm` shares its exact statement with another label. An evaluation that samples targets uniformly and reports success will be measuring duplicate-lookup on about a sixth of its sample and, since duplicates are by construction the easiest targets, on a far larger share of its *successes*.

5.4 A corrected protocol

Definition 5.1 (Admissible index)**.** Let φ be a target at ordinal position k. The index available to the searcher must exclude every label ℓ with position $< k$ such that the conclusion of ℓ is a substitution instance of φ, or φ is a substitution instance of the conclusion of ℓ, under a renaming of variables.

Definition 5.2 (Non-trivial reach)**.** The *non-trivial reach* of a prover on a corpus is the number of targets it proves under the admissible index, from a stated sample, within a stated budget.

Excluding only exact duplicates is not enough — `falanfal` would survive that test, since $(F. \wedge F.) \leftrightarrow F.$ is not literally $(\varphi \wedge \varphi) \leftrightarrow \varphi$ — which is why the definition is stated in terms of substitution instances in both directions.

Current status. Predator_7.1's non-trivial reach on `set.mm` is **not yet measured**, and on the evidence of the four certificates the honest prior is that it is small, possibly zero. The verified figure of "four theorems proved" should be read as "four theorems located".

Part II — Cross-branch reuse rate theory

6 The preorder on SICs, and which one is meant

Revision 1 of this document argued that the settlement-region programme — classifying a conjecture c by the upward-closed set of machines that settle it — is vacuous, on the ground that every SIC simulates every other. That argument was wrong, and the error is instructive enough to record in full.

6.1 Weak simulation does collapse

v45 proves the following.

> **Theorem** (`thm:weakcollapse`). *Every SIC weakly simulates into every other SIC.*

The proof takes the witness relation to be $R := S_M \times S_N$, the total relation, and observes that the closure and anchoring conditions hold because membership in R is automatic. The theorem is correct. But note what the proof does: it satisfies the definition by making the witness carry no information. Weak simulation never requires N to track M's states, preserve its theorems, or match its halting behaviour. `thm:weakcollapse` is therefore best read as a *degeneracy result about a definition*, not as a fact about SICs, and v45's own table describes $S_M \times S_N$ as the "witness for universal weak simulation" — an object whose role is to exhibit the collapse.

6.2 Clocked simulation does not

v45's principal relation is different, and it is the one written $\preceq$ without adornment:

> "Because this is the principal simulation relation in the remainder of the paper, the unadorned notation $M \preceq N$ abbreviates $M \preceq_{\mathrm{clk}} N$."

Definition 6.1 (Clocked deterministic simulation, v45)**.** An *admissible clock* is a nondecreasing unbounded $\rho \in [\mathbb{N}^+ \to \mathbb{N}^+]$ with $\rho(1) = 1$. A *clocked deterministic simulation* from M to N is a pair (f, ρ) with $f \in [S_M \to S_N]$ **injective**, ρ an admissible clock, and for every $p \in S_M$ and $m \in \mathbb{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), \rho(m)), \qquad H_M(p, m) = H_N(f(p), \rho(m)).$$

This is a strong condition: exact state correspondence and exact halting correspondence, up to a time rescaling. v45's accompanying remark is explicit that clocked simulation "repairs timing rigidity, not cardinality rigidity" and that "lossy many-to-one comparisons remain state abstractions rather than simulations".

Proposition 6.2 (Non-collapse of $\preceq_{\mathrm{clk}}$)**.** *There exist SICs M and N with $M \not\preceq_{\mathrm{clk}} N$.*

Proof. Two results of v45 combine. `prop:statespacesaturation` gives $S_M = \mathcal{P}(Y_M)$ for every SIC M over Y_M, since the stage-one axiom $C(\Gamma, 1) = \Gamma$ makes every subset of Y_M a state. `prop:cardinalityobstruction` then observes that an injective $f : S_M \to S_N$ is an injection $\mathcal{P}(Y_M) \to \mathcal{P}(Y_N)$, so in the finite case $2^{|Y_M|} \le 2^{|Y_N|}$ and hence $|Y_M| \le |Y_N|$.

Injectivity of f is part of Definition 6.1, and the clock ρ plays no role in the cardinality argument, so the obstruction applies to $\preceq_{\mathrm{clk}}$ verbatim. Taking $Y_M = \{a, b\}$ and $Y_N = \{c\}$ with both SICs static and never halting — v45's own example — gives $M \not\preceq_{\mathrm{clk}} N$, since no injection $\mathcal{P}(\{a, b\}) \to \mathcal{P}(\{c\})$ exists. □

Corollary 6.3 (The settlement programme is not vacuous)**.** *The quotient* (SIC/ $\sim$, $\preceq_{\mathrm{clk}}$) *is a partial order with more than one element. Consequently, for a conjecture c, the settlement region U_c upward-closed under $\preceq_{\mathrm{clk}}$ is a nontrivial construction, and the questions of whether U_c has a least element, several minimal elements, or is generated by an antichain are open rather than degenerate.*

Remark 6.4 (the correction, and what remains of the criticism)**.** Revision 1's conclusion is reversed. What survives is a separate point, which does not depend on which simulation relation is used: if Settles(X, c) means only that X eventually outputs a proof of c or of $\neg c$ by unbounded search from a *fixed* theory T, then whether such a search halts depends on T and c alone, and U_c takes only the values $\varnothing$ and everything. Non-degeneracy of the preorder does not by itself repair that; the settlement predicate must additionally be **resource-bounded**, so that "X settles c" means "X settles c within its own budget". With both corrections in place — a non-collapsing preorder and a bounded predicate — the programme has content.

v45 supplies the machinery for the bounded reading as well. Its p-simulation relation $\preceq_{\mathrm{p}}$ compares proof systems by a polynomial-time proof-to-proof map, and `prop:psimbound` gives the transfer $s_{F'}(\Gamma) \le q(s_F(\Gamma))$ for a polynomial q. v45 notes that $\preceq_{\mathrm{p}}$, unlike weak simulation, "does not collapse universally, because polynomial-time computability of h is a genuine restriction once size, rather than merely stage number, is being tracked". That is the same observation as the resource-bound requirement above, arrived at from the proof-complexity side.

Remark 6.5 (a caution about five relations)**.** v45 defines at least five comparison notions: weak, synchronous, and clocked simulation, invertible recoding, and p-simulation. Only the first collapses. Any statement of the form "the preorder on SICs is X" is therefore ambiguous, and Revision 1 fell into exactly that ambiguity. The provenance tables of Section 17 list each relation with its origin for this reason.

7 Quantities

Fix a formal system $F = (x, y, z)$ with direct-consequence operator D_F, a decidable $Y \subseteq y$, and finite $\Gamma \subseteq Y$. From v45:

$$\|\varphi\|_F(\Gamma) = \min\{|\pi| : \pi \text{ a proof from } \Gamma \text{ with final line } \varphi\},$$

$$C_{F,Y}(\Gamma, 1) = \Gamma, \qquad C_{F,Y}(\Gamma, m{+}1) = D_F(C_{F,Y}(\Gamma, m)),$$

and, for a target-search ATP M for φ, $\tau_M(\Gamma, \varphi) = \min\{m : \varphi \in C(\Gamma, m)\}$.

Remark 7.1 (which machine realises the horizon)**.** v45 distinguishes two canonical machines that are easy to conflate, and Revision 1 conflated them.

- $E_{F,Y}$, the canonical rule-enumeration SIC (`def:canonicalenum`), has $H_{F,Y} \equiv 0$. **It never halts**, so τ is not defined for it as a target-search count.
- $B_{F,Y,\varphi}$, the canonical breadth-first target-search SIC, has $C^{\mathrm{bf}}(\Gamma, m) = \{\psi : \|\psi\|_F(\Gamma) \le m\}$ and halts exactly when $\|\varphi\|_F(\Gamma) \le m$.

The Proof-Horizon Theorem (`thm:proofhorizon`, part 2) states that $B_{F,Y,\varphi}$ halts iff $\Gamma \vdash_F \varphi$, with first halting stage exactly $\|\varphi\|_F(\Gamma)$; the corollary rewrites this as $\tau_{B_{F,Y,\varphi}}(\Gamma, \varphi) = \|\varphi\|_F(\Gamma)$. The canonical *target-search* machine $E_{F,Y,\varphi}$ of `def:canonicaltarget` does halt, and has the same stage sets, so the identity holds for it too; the machine for which it fails is the non-halting enumerator $E_{F,Y}$.

Remark 7.2 (the qualifier on the lower bound)**.** v45's corollary to the Branch-Covering Theorem reads: "no target-search SIC *based solely on shortest proof length* can reach φ earlier than that stage". The italicised qualifier is not decoration. A machine given side information — a precomputed cache, an oracle, a compiled derived rule — is not based solely on shortest proof length, and the Conservative Compilation Theorem is precisely the construction of such a machine. Statements in this document of the form "nothing beats the horizon" carry the qualifier.

To the transaction count add the *materialisation count* μ, the number of distinct formulas built before halting. Unlike τ, which Remark 7.1 bounds below by the horizon, μ is bounded below by nothing, and is therefore where a search policy operates.

8 The settlement machine

Definition 8.1 (Settlement machine)**.** Let $\Gamma \subseteq Y$ and let $c \in Y$ with $\neg c \in Y$. The *settlement machine* $M^{\otimes} = (C^{\otimes}, H^{\otimes})$ maintains two branches: A pursuing c and B pursuing $\neg c$, each a policy-directed search over Y. Odd stages advance A and even stages advance B. Both read from and write to a shared *lemma store* Λ. Define

$$C^{\otimes}(\Gamma, m) := \Lambda_m \cup A_m \cup B_m, \qquad H^{\otimes}(\Gamma, m) := \begin{cases} 1, & c \in C^{\otimes}(\Gamma, m) \text{ or } \neg c \in C^{\otimes}(\Gamma, m), \\ 0, & \text{otherwise,} \end{cases}$$

where A_m, B_m are the branch states and Λ_m the store at stage m, with the stipulation that all three are frozen once $H^{\otimes} = 1$.

Proposition 8.2 ($M^{\otimes}$ is a SIC)**.** *$M^{\otimes}$ satisfies the six conditions of* `def:sic`*.*

Proof. Conditions (1) and (2) are the typing of $C^{\otimes}$ and $H^{\otimes}$, which hold by construction. (3): at stage 1 neither branch has acted and $\Lambda_1 = \varnothing$, $A_1 = B_1 = \Gamma$, so $C^{\otimes}(\Gamma, 1) = \Gamma$. (4): branch states and the store only ever grow, being closed under adjoining new formulas and never removing any, so $C^{\otimes}$ is monotone in m. (5): $H^{\otimes}(\Gamma, m) = 1$ requires c or $\neg c$ to lie in $C^{\otimes}(\Gamma, m)$, and by monotonicity it lies in $C^{\otimes}(\Gamma, m+1)$ as well, so halting persists. (6): the freezing stipulation gives $C^{\otimes}(\Gamma, m+1) = C^{\otimes}(\Gamma, m)$ once $H^{\otimes} = 1$. □

Remark 8.3 (why a new count is needed)**.** $M^{\otimes}$ is **not** a target-search ATP for c in the sense of `def:targetsearch`, which requires that $H(\Gamma, m) = 1$ imply $\varphi \in C(\Gamma, m)$. If $\neg c$ is the provable side, $M^{\otimes}$ halts with $c \notin C^{\otimes}$, violating that clause. Revision 1 treated $M^{\otimes}$ as a target-search ATP and applied τ to it, which was an error. The repair is to extend the count rather than to distort the machine.

Definition 8.4 (Settlement count)**.** For a SIC M over Y and $c \in Y$ with $\neg c \in Y$, define

$$\sigma_M(\Gamma, c) := \min\{m \in \mathbb{N}^+ : c \in C(\Gamma, m) \text{ or } \neg c \in C(\Gamma, m)\},$$

and $\sigma_M(\Gamma, c) := \infty$ if no such m exists.

Proposition 8.5 (σ extends τ)**.** $\sigma_M(\Gamma, c) = \min\big(\tau_M(\Gamma, c),\ \tau_M(\Gamma, \neg c)\big)$ *whenever the right-hand side is defined, and if Γ is consistent then at most one of the two is finite.*

Proof. The first claim is immediate from the definitions, since the minimum over a union of two conditions is the minimum of the two minima. For the second, if both $\Gamma \vdash_F c$ and $\Gamma \vdash_F \neg c$ then Γ is inconsistent. □

Definition 8.6 (Origin, crossing, reuse rate)**.** Each $\lambda \in \Lambda$ carries an origin $\mathrm{o}(\lambda) \in \{A, B\}$: the branch that closed it. Write Λ_m for the store at stage m. Say λ is

- *available-crossed* by stage m if λ lies in the candidate set of the branch other than $\mathrm{o}(\lambda)$ at some stage $\leq m$;
- *use-crossed* by stage m if λ occurs as a premise of a step actually taken by the branch other than $\mathrm{o}(\lambda)$ at some stage $\leq m$.

The corresponding *cross-branch reuse rates* are

$$\kappa^{\mathrm{av}}_m = \frac{|\{\lambda \in \Lambda_m \text{ available-crossed by } m\}|}{|\Lambda_m|}, \qquad \kappa^{\mathrm{us}}_m = \frac{|\{\lambda \in \Lambda_m \text{ use-crossed by } m\}|}{|\Lambda_m|},$$

with both set to 0 when $\Lambda_m = \varnothing$. Write $\kappa^{\bullet} = \limsup_m \kappa^{\bullet}_m$. Plainly $\kappa^{\mathrm{us}} \leq \kappa^{\mathrm{av}}$.

Definition 8.7 (Cross-branch insertion)**.** $M^{\otimes}$ has *cross-branch insertion* if, whenever a branch closes a ground lemma λ, the nullary derived rule d_λ — empty premise tuple, value λ — is adjoined to the rule set, making λ available to both branches at unit cost thereafter.

9 First result: reuse is a property of the policy

Theorem 9.1 (Canonical degeneracy)**.** *Let $M^{\otimes}$ be a settlement machine in which both branches run the unrestricted operator D_F, with no policy targeting. Then for every Γ and every c:*

(i) $A_m = B_m = C_{F,Y}(\Gamma, m)$ for all m, so the two branches are indistinguishable and the origin function o is not determined by the run;

(ii) under any convention assigning origins, $\kappa^{\mathrm{av}}_m = 1$ whenever $\Lambda_m \neq \varnothing$;

(iii) neither quantity depends on c.

Consequently κ is not an invariant of the pair (Γ, c): any value other than 1 is produced by the policy, and $1 - \kappa^{\mathrm{av}}$ measures the degree to which the policy has separated the branches.

Proof. (i) D_F maps a state to the set of its direct consequences and does not mention a target. Both branches therefore apply the same operator to the same initial state, and induction on m gives $A_m = B_m = C_{F,Y}(\Gamma, m)$. Any formula entering at stage m enters both simultaneously, so nothing in the run distinguishes which branch produced it.

(ii) Fix any origin convention and let $\lambda \in \Lambda_m$ with, say, $\mathrm{o}(\lambda) = A$. By (i), $\lambda \in B_m$, so λ lies in the candidate set of branch B at stage m, and is available-crossed. As λ was arbitrary, $\kappa^{\mathrm{av}}_m = 1$.

(iii) Neither argument mentions c. A quantity taking the same value for every conjecture distinguishes no conjectures. □

Remark 9.2 (why the theorem is about availability, not use)**.** The corresponding claim for κ^{us} is false. A lemma may sit in both branches' states and never be a premise of any admissible rule instance — in condensed detachment, a formula that unifies with no antecedent is available forever and used never. So $\kappa^{\mathrm{us}} < 1$ is possible even for the canonical machine, and the degeneracy is properly a statement about what the branches can *see*. Revision 1 asserted $\kappa = 1$ without the distinction, which was wrong.

Corollary 9.3 (The two degenerate regimes)**.** $\kappa^{\mathrm{av}} = 1$ *is the signature of no targeting: the machine is breadth-first and the branch labels are decoration.* $\kappa^{\mathrm{us}} = 0$ *is the signature of total separation: no lemma is ever consumed across the divide, the two searches are independent, and settlement costs a factor of* 2 *in* σ *while buying only the ability to detect a refutation. Both extremes are uninformative; the empirical question is where a real policy falls between them.*

Remark 9.4 (position in the v45 scheme)**.** The Proof-Horizon Theorem makes τ theory-determined and, subject to the qualifier of Remark 7.2, policy-invariant. The materialisation count μ is the complementary quantity: policy-determined, bounded below by nothing. Theorem 9.1 places κ on the policy side of that divide, for the same underlying reason — both count what the machine *did*, not how deep it had to go.

10 Second result: cross-branch reuse is online compilation

Theorem 10.1 (Reuse as compilation)**.** *Let* $M^{\otimes}$ *be a settlement machine with cross-branch insertion, and let* M^{sep} *be the same machine with the shared store removed, so the branches are independent. Then:*

(i) the augmented system F^D*,* $D = \{d_\lambda\}$*, is conservative over* F*;*

(ii) $\sigma_{M^{\otimes}}(\Gamma, c) \;\le\; \sigma_{M^{\mathrm{sep}}}(\Gamma, c) \;\le\; 2\min\big(\tau_{M_A}(\Gamma, c), \tau_{M_B}(\Gamma, \neg c)\big)$*, where* M_A, M_B *are the single-target machines on the same policy;*

(iii) writing $\Delta(c) := \sigma_{M^{\mathrm{sep}}}(\Gamma, c) - \sigma_{M^{\otimes}}(\Gamma, c)$ *for the* settlement dividend*,* $\Delta(c) \ge 0$*; and* $\Delta(c) > 0$ *only if some lemma is use-crossed, so* $\kappa^{\mathrm{us}} = 0 \implies \Delta(c) = 0$*.*

Proof. (i) Each inserted λ was closed by a branch, so $\Gamma \vdash_F \lambda$, and d_λ is a derived rule in the sense used by v45's conservativity proposition: replace each line produced by d_λ with an F-proof of λ from Γ. Since any F^D-proof is finite, the replacement terminates and yields an F-proof of the same conclusion. Nothing here depends on which branch produced λ, which is the point: soundness is indifferent to intent.

(ii) The right inequality is the cost of alternation: each stage of M^{sep} advances one branch, so two of its stages contain at least one stage of whichever branch halts first, and Proposition 8.5 identifies σ with the smaller of the two τ's. For the left inequality, $M^{\otimes}$ and M^{sep} differ only in that the former has the additional nullary rules available. Adjoining a nullary rule can only enlarge $D_F(S)$ for every S, so induction on m gives $C^{\otimes}(\Gamma, m) \supseteq C^{\mathrm{sep}}(\Gamma, m)$, whence the least stage containing c or $\neg c$ is no larger.

(iii) Non-negativity is immediate from (ii). If $\Delta(c) > 0$ then some stage was saved; since the machines differ only in the rules d_λ, some d_λ must have fired in the branch that halted. A rule d_λ is available to that branch only by insertion, so λ was closed by the other branch and consumed by this one, i.e. use-crossed. Contrapositively, if no lemma is use-crossed then no d_λ fires across the divide, the two machines' stage sequences agree, and $\Delta(c) = 0$. □

Remark 10.2 (the converse fails)**.** $\Delta(c) > 0$ implies a use-crossed lemma, but not conversely: a crossed lemma that shortens a branch's proof by zero stages — because an equally short route existed — contributes nothing. So $\kappa^{\mathrm{us}} > 0$ is necessary and not sufficient for a positive dividend. Revision 1 asserted an equivalence here.

Corollary 10.3 (No wasted branch)**.** *Suppose* Γ *is consistent. By Proposition 8.5 at most one of* $c, \neg c$ *is provable, so at least one branch never halts. Nevertheless every lemma that branch closes*

is a theorem of Γ*, is sound to insert by (i), and is available to the other branch. The non-halting branch is a lemma generator, and its contribution to the halting branch is exactly* $\Delta(c)$*.*

Remark 10.4 (what this makes of "open-mindedness")**.** The intuition motivating settlement search was psychological: a searcher spending equal effort on c and $\neg c$ is unbiased, whereas a mathematician who fails to prove c for a decade and begins believing $\neg c$ has merely run out of patience. Corollary 10.3 replaces that intuition with an inequality. When $\Delta(c) > 0$ the unbiased search is not merely fairer than the biased one; it reaches the provable side in *fewer* stages than the machine that separated the branches. The refutation attempt pays for itself, in the currency v45 already uses.

The qualification should not be lost. When $\kappa^{\text{us}} = 0$ the dividend is zero and settlement is a pure factor-of-two loss. Whether $\kappa^{\text{us}} > 0$ for any real policy on any real corpus is an empirical question, stated as Problem 12.1.

11 Third result: chain deficits and speedup

11.1 What the speedup principle actually requires

v45's Abstract Asymptotic Speedup Principle has three hypotheses: (1) $\tau_{E_{F^D,Y,\varphi_n}}(\Gamma_n, \varphi_n) \le p(n)$; (2) **every** proof of φ_n from Γ_n **in** F has length at least $q(n)$; (3) $q(n)/p(n)$ unbounded.

Hypothesis (2) is a statement about the formal system, not about any policy. It is a proof-length lower bound of exactly the kind Part III will show is currently unobtainable in strong systems. Revision 1's Theorem 7.2 claimed to witness this principle on the strength of a hypothesis about a policy, which does not suffice. The theorem below is therefore stated as what it actually gives — a policy-relative separation — with the missing ingredient named.

Definition 11.1 (Chain deficit)**.** A family (Γ_n, c_n) with $\Gamma_n \vdash_F c_n$ for every n — so that c_n is the provable side and, by Proposition 8.5, $\sigma^{\otimes} = \tau^{\otimes}$ for the settlement machine — has a *chain deficit* (ℓ, g, h) under policy Σ if there are formulas $\lambda_1^n, \dots, \lambda_{\ell(n)}^n$ with:

(a) **Chain, not layer.** λ_k^n occurs as a premise in every derivation of λ_{k+1}^n from Γ_n of length below $g(n)$.

(b) **Cheap on the refutation side.** Σ pursuing $\neg c_n$ closes the whole chain within $O(\ell(n))$ stages.

(c) **Expensive on the proof side.** Σ pursuing c_n does not close $\lambda_{\ell(n)}^n$ before stage $g(n)$.

(d) **On the proof.** Some Σ-reachable shortest proof of c_n cites $\lambda_{\ell(n)}^n$, and given it, c_n follows in $h(n)$ further stages.

Theorem 11.2 (Deficit implies policy-relative speedup)**.** *Suppose* (Γ_n, c_n) *has a chain deficit* (ℓ, g, h) *under* Σ*. Then the settlement machine with cross-branch insertion satisfies*

$$\sigma^{\otimes}(\Gamma_n, c_n) = O(\ell(n) + h(n)), \qquad \tau^{\Sigma}(\Gamma_n, c_n) \;\ge\; g(n) + h(n),$$

and therefore

$$\frac{\tau^{\Sigma}(\Gamma_n, c_n)}{\sigma^{\otimes}(\Gamma_n, c_n)} = \Omega\!\left(\frac{g(n) + h(n)}{\ell(n) + h(n)}\right).$$

If $\ell(n), h(n) = O(n)$ *and* $g(n) = \omega(n)$ *the ratio is unbounded, and the settlement machine is asymptotically faster than the single-target machine* running the same policy Σ*.*

Proof. By (b), branch B closes $\lambda_1^n, \dots, \lambda_{\ell(n)}^n$ within $O(\ell(n))$ of its own stages, hence within $O(\ell(n))$ settlement stages after the factor-2 alternation of Theorem 10.1(ii). By cross-branch insertion each λ_k^n becomes a nullary rule as it is closed, so $\lambda_{\ell(n)}^n$ is available to branch A at unit cost from that point. By (d), A reaches c_n in $h(n)$ further stages, giving $\sigma^{\otimes} = O(\ell(n) + h(n))$.

For the lower bound, the single-target machine runs Σ against c_n only. By (c) it does not hold $\lambda_{\ell(n)}^n$ before stage $g(n)$. By (a) and (d), reaching c_n before stage $g(n) + h(n)$ would require either closing $\lambda_{\ell(n)}^n$ earlier, excluded by (c), or a Σ-reachable route avoiding it, excluded by (d). Dividing gives the ratio. □

Remark 11.3 (what would upgrade this to `thm:asympspeedup`)**.** Theorem 11.2 compares two machines running the same policy. v45's principle compares an augmented canonical machine against *line-by-line search in* F, and its hypothesis (2) requires $\|c_n\|_F(\Gamma_n) \geq q(n)$ — a lower bound over all F-proofs, not over those a policy happens to find. To upgrade, condition (c) must be replaced by

(c') every F-proof of c_n from Γ_n has length at least $g(n) + h(n)$,

whereupon hypotheses (1)–(3) hold with $p(n) = O(\ell(n) + h(n))$ and $q(n) = g(n) + h(n)$.

Condition (c') is a genuine proof-complexity lower bound. On a fragment small enough for breadth-first search to terminate — the 287-state condensed detachment fragment of Predator_5, say — it can be *computed* for each small n by exhaustive search, since breadth-first search returns the true shortest-proof length. That is the practical route to verifying a candidate family, and it does not scale, which is the honest statement of the difficulty.

Corollary 11.4 (Why the previous family could not separate)**.** *Let $D = \{d_1, \dots, d_n\}$ be a* layer*: each d_k derivable from Γ_n without the others. Then condition (a) fails outright, since no d_k is a premise of any d_{k+1}, so $g(n) = O(\ell(n))$ and the ratio in Theorem 11.2 is $\Theta(1)$. This is the structural reason an earlier attempt in this program produced $p(d) = d$ and $q/p \to 1$. Superlinear g requires the links to be premises of one another — which is what makes a chain a chain rather than a layer.*

11.2 What a chain looks like, and what a layer looks like

The distinction Corollary 11.4 turns on is easier to see than to state. Figure 1 is the lemma graph of a worked target from this program: the theorem that the set of infinitesimals I in a nonstandard extension is equipollent to $\mathbb{R}$, decomposed into seventeen lemmas.

Both structures are present, which is what makes the example useful. A settlement search that obtained L11 from the refutation branch would save the whole prefix L1, L3, L4 on the proof branch; a search that obtained L15 would save one step, because L14 is independently cheap. Condition (a) of Definition 11.1 is precisely the demand that the crossed lemma sit on a spine rather than on a frond, and $g(n)$ grows superlinearly only when the spine does.

The graph also supplies a concrete target for Definition 5.2: L17 has depth 6 and seventeen antecedents, none of which is a duplicate of it under Definition 5.1. It is therefore an admissible non-trivial target of exactly the kind Part I found to be missing from the four verified certificates.

Conjecture 11.5 (Chain deficits exist)**.** There is a recursive family (Γ_n, c_n) over the condensed-detachment fragment, with $\Gamma_n = \{K, S, W\}$ augmented by n auxiliary formulas, admitting a chain deficit with $\ell(n) = \Theta(n)$, $h(n) = O(1)$ and $g(n) = \Omega(n^2)$ under the size-ordered policy of Predator_5.

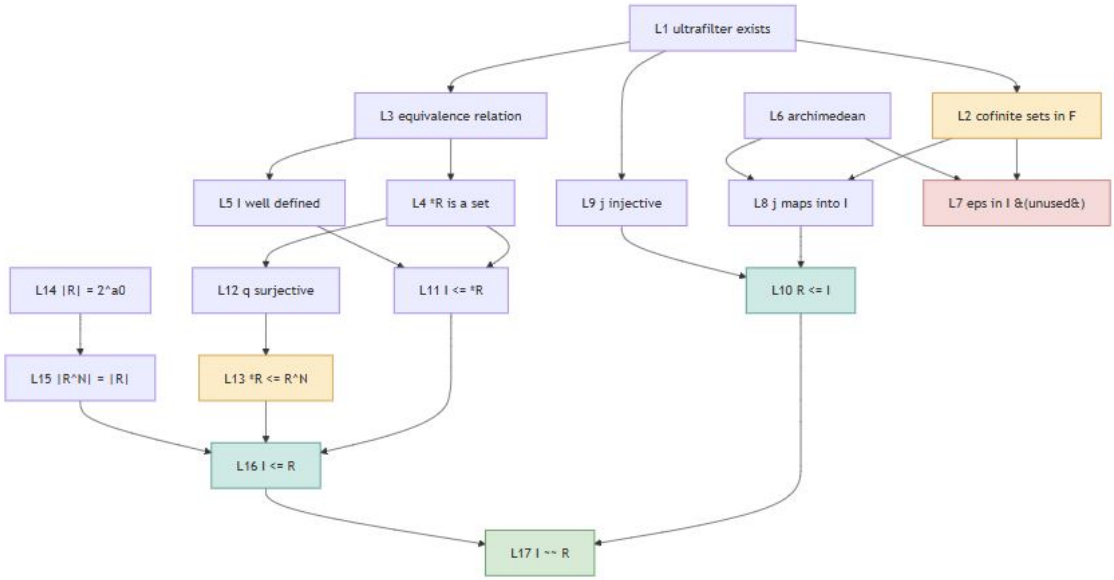


Figure 1: Lemma graph for $|I| = |\mathbb{R}|$. The spine L1 → L3 → L4 → L11 → L16 → L17 is a *chain* in the sense of Definition 11.1(a): each link is a premise of the next, and no link can be skipped. The strands L14 → L15 and L12 → L13, hanging off L4 and L5, are short parallel branches — closer to a *layer*, and individually skippable.

12 Measurement protocol

Problem 12.1 (The number nobody has)**.** Measure κ^{us} for a real policy on a real corpus.

The measurement is cheap relative to what already exists, because the expensive object — the parsed index — is shared:

1. **One index, shared.** Building it costs 8–12 seconds and dominates short runs.
2. **Two goal stacks**, alternating on expansion, odd to c and even to $\neg c$.
3. **Cross-branch insertion.** When either branch closes a subgoal the result is a ground theorem with no free metavariables; adjoin it to the shared index as a new closer. By Theorem 10.1 this is online compilation.
4. **Log origins and uses.** κ^{us} is then a ratio of two counters, and κ^{av} of two more.

Remark 12.2 (a distinction the implementation must respect)**.** The branches cannot share *substitutions*: their metavariable bindings are independent and unifying across them would be unsound. What they share is *closed lemmas* — ground, fully determined, no free metavariables. An implementation that shares the substitution store rather than the lemma store will produce proofs the CV rejects, and that rejection will be correct.

For a test set, any `set.mm` theorem of the form `|- -. phi` supplies a conjecture φ whose refutation is known reachable, so branch B terminates and κ is well defined on a finite run. Targets must be filtered through the admissible index of Definition 5.1, or the measurement will be contaminated by the duplicate-lookup effect of Section 5.

Part III — Lower bounds on the settlement length of P vs NP

13 The certificate verifier is an NP verifier

The connection is not an analogy. NP is *defined* by certificate verification: $L \in \mathrm{NP}$ iff there are a polynomial-time V and a polynomial p with $x \in L \iff \exists w,\ |w| \le p(|x|),\ V(x,w) = 1$. The file `metamath_cv.py` is such a V.

Definition 13.1 (Verification transcript)**.** For a Metamath proof π under database Σ, the *transcript* is the sequence of stack contents materialised during verification; its size $T(\pi)$ is the total number of symbols across all stack entries pushed.

Proposition 13.2 (Verification is polynomial in the transcript)**.** *There is an algorithm deciding, given Σ, π and φ, whether π is a proof of φ from Σ, in time polynomial in $|\Sigma| + T(\pi)$.*

Proof sketch. Each step pushes a hypothesis or applies an assertion. Applying an assertion requires matching the top k stack entries against the assertion's hypotheses — linear in their combined size — computing the induced substitution, and applying it to the conclusion; each is polynomial in the sizes of the entries involved, and every such entry is part of the transcript by definition. Disjoint-variable checks are quadratic in the number of variables of the frame. Summing over steps gives the bound. □

Remark 13.3 (why the transcript and not the proof)**.** The bound is stated in $T(\pi)$ rather than $|\pi|$ deliberately. Substitution can square the size of a formula at a single step, so an n-step proof can materialise formulas of size exponential in n, and verification time is not in general polynomial in the length of the proof *file*. For `set.mm` in practice the blowup does not occur — the corpus verifies in under 200 seconds — but a theoretical statement must respect the worst case. This is the same distinction v45 draws between $\|\varphi\|_F(\Gamma)$ and $\|\varphi\|_F^{\mathrm{size}}(\Gamma)$.

Proposition 13.4 (Bounded provability is in NP)**.** *Fix a database Σ. Then* $\textsc{Short-Proof}_\Sigma = \{(\varphi, 1^n) : \exists \pi,\ T(\pi) \le n,\ \pi \text{ proves } \varphi \text{ from } \Sigma\} \in \mathrm{NP}$*, with π the certificate and `metamath_cv.py` the verifier.*

Proof. The certificate satisfies $T(\pi) \le n$, hence has size at most n, polynomial in the input length by virtue of the unary encoding of the bound. Proposition 13.2 makes verification polynomial. □

Remark 13.5 (the reflexive turn)**.** Settling P vs NP inside `set.mm` means exhibiting a formula $\varphi \equiv$ "P $\neq$ NP" together with a certificate a polynomial-time verifier accepts. The certificate verifier is simultaneously the instrument of the project and an instance of the object quantified over in the statement being settled. This obstructs nothing, but it does mean that any claim about the difficulty of producing the certificate is a claim about an NP search problem, and v45's `def:automatizable` is the right frame: what is being asked is whether the proof system is automatizable at the relevant length.

Remark 13.6 (computation is proof, and where it stops)**.** v45's Turing-machine appendix supplies a bridge worth stating here. Its configuration formal system F_T takes the one-step function of a deterministic machine T as its single inference rule, and `prop:reachability` identifies provability

in F_T with reachability of configurations. The accompanying remark observes that a run of T is a proof *with proof length equal to the number of steps taken plus one*. Hence in F_T,

$$\|\cdot\|_{F_T} = \text{running time}, \qquad \tau = \text{time complexity}.$$

This makes deterministic time expressible in the SIC vocabulary directly.

It does not yet make NP expressible. The rule set is $z_T = \{\text{step}_T\}$, a single unary rule which is a *function*; the appendix has no nondeterministic analogue, so no branching machine can be written as an F_T. Two repairs are available: make z_T relational, so a configuration has several successors, or go through certificates as in Proposition 13.4. The second is what the CV already instantiates, and is the cheaper of the two.

14 Four routes to a lower bound

Write $\|\text{P} \neq \text{NP}\|$ for the number of logical steps in a shortest `set.mm`-style proof settling the conjecture, on the understanding that the database must first be extended to make the statement expressible.

14.1 Route 1: counting. Fails, and instructively

Proposition 14.1. *Let L be the number of labels available. The number of proofs of length at most n is at most $L^{n+1}/(L-1)$; hence at most that many statements have proofs of length $\leq n$.*

With $L = 50{,}572$ this gives, at the median `set.mm` raw proof length of 173, a bound near 10^{821} — against 43,836 distinct statements in the corpus. The counting argument imposes no constraint at the lengths in question.

The failure is structural. Counting shows that *most* statements require long proofs; it is a counting argument and P $\neq$ NP is a single statement. Any particular statement may be the exception, and no counting principle can say otherwise. Recorded because it is the first thing one tries.

14.2 Route 2: the proof horizon. Real, but bounds the machine

The Proof-Horizon Theorem gives $\tau_{B_{F,Y,\varphi}}(\Gamma, \varphi) = \|\varphi\|_F(\Gamma)$, and the corollary to the Branch-Covering Theorem gives that no target-search SIC *based solely on shortest proof length* reaches φ earlier.

That is a bound on the *machine*: whatever $\|\text{P} \neq \text{NP}\|$ turns out to be, no policy of that kind beats it, so no amount of cleverness in Predator converts a long proof into a short search. It rules out a class of hopes. It takes $\|\varphi\|$ as given and says nothing about its size. And, per Remark 7.2, it does not apply to machines with compiled side information — which is exactly why the Conservative Compilation Theorem is not in conflict with it.

14.3 Route 3: the barriers. Bound the content, not the length

Barrier	Source	Excludes
Relativization	Baker–Gill–Solovay 1975	proofs holding under all oracles
Natural proofs	Razborov–Rudich 1997	large constructive circuit lower bounds
Algebrization	Aaronson–Wigderson 2009	proofs surviving algebraic oracles

Each converts into a statement about proof *content*: any proof of P $\neq$ NP must contain a non-relativizing step, an ingredient that is not natural in the Razborov–Rudich sense, and so on. Converting content into length requires a lower bound on the cost of *expressing* a barrier-crossing ingredient in the formal system, and no such bound is known. The barriers say the proof cannot have certain shapes; they do not say how many lines the permitted shapes require.

14.4 Route 4: the definitional floor

Definition 14.2 (Definitional floor)**.** For a statement φ not expressible in a database Σ, the *definitional floor* $D_\Sigma(\varphi)$ is the least number of new logical assertions that must be added to Σ before φ becomes expressible and its constituent notions usable — that is, before the defining assertions are accompanied by the elimination and congruence lemmas without which no proof can manipulate them.

Proposition 14.3. $\|\varphi\|_\Sigma \geq D_\Sigma(\varphi)$.

Immediate; its value lies entirely in whether D can be estimated. For P $\neq$ NP over `set.mm` it can, because the starting point is unusually clean.

Proposition 14.4 (`set.mm` contains no complexity theory)**.** *The current `set.mm` contains no machine model, no time-bounded computation, and no complexity class. The string `Turing` occurs zero times in the file. No label matches any of `turing`, `machin`, `halt`, `comput`, `decid`, `recurs` or `complex` as a substring, except five topology labels (`clsneircomplex`, `neicvgrcomplex`, `ntrclsrcomplex`, `ntrneircomplex`, `ivthALT`) whose match is coincidental.*

So $D_{\texttt{set.mm}}$(P $\neq$ NP) is the entire cost of building computability theory and then complexity theory on top of it. Note that this is a *formalisation* cost, not a conceptual gap: v45's appendix already supplies the machine model and the identification of runtime with proof length (Remark 13.6). What is missing is a `set.mm` development of it.

That cost can be calibrated against buildups `set.mm` has already paid for. Writing dep(φ) for the number of logical assertions in the transitive closure of φ's citations:

Theorem	Ordinal position	dep	dep/position
`pm2.01`	189	37	0.20
`simpl`	486	44	0.09
`falim`	1,584	80	0.05
`sbth`	9,083	1,299	0.14
`canth2`	9,116	1,608	0.18
`zorn`	10,489	2,707	0.26
`ruc`	16,297	3,585	0.22
`sqrt2irr`	16,303	3,914	0.24
`pythag`	26,957	7,568	0.28
`fta`	27,219	7,839	0.29
`bpos`	27,432	8,271	0.30

The pattern is stable: a theorem at the depth of the fundamental theorem of algebra or Bertrand's postulate rests on roughly 8,000 logical assertions, and the ratio converges towards 0.3.

Estimate (not a theorem). Computability theory — a machine model, its semantics, the recursion theorem, the halting problem — is comparable in definitional depth to the construction of the reals, which `set.mm` reaches near position 16,000 with roughly 3,600 dependencies. Complexity theory sits above it: time-bounded computation, polynomial bounds, the classes, and the closure lemmas needed to reason about them. Taking the calibration at face value,

$$D_{\texttt{set.mm}}(\mathrm{P} \neq \mathrm{NP}) \;\sim\; 10^4 \quad \text{logical assertions,}$$

hence $\|\mathrm{P} \neq \mathrm{NP}\| \gtrsim 10^4$ *before a single step of the proof proper.* The estimate is soft in its constant and firm in its order of magnitude; what is rigorous is Proposition 14.3 together with Proposition 14.4, which is checkable in one command.

15 What is actually known about proof length lower bounds

Proof system	Best known lower bound	Status
Resolution	exponential (Haken 1985, PHP)	settled
Bounded-depth Frege	exponential (Ajtai 1988; later improvements)	settled
Frege	**none superpolynomial, for any tautology**	open
Extended Frege	**none superpolynomial, for any tautology**	open
ZFC / `set.mm`	**nothing**	open

The gap between the third row and the first is the central open problem of proof complexity. Nobody can exhibit a single tautology requiring superpolynomial Frege proofs, and `set.mm` is a system far stronger than Frege.

No unconditional, nontrivial lower bound on $\|\mathrm{P} \neq \mathrm{NP}\|$ in a strong system is currently obtainable. Producing one would itself be a major result in proof complexity, independent of anything it implied about P vs NP. The definitional floor is not an exception: it bounds the cost of *stating* the conjecture, which is a different and much easier quantity than the cost of proving it.

Remark 15.1 (where this belongs in v45)**.** v45 already contains the relevant apparatus — `def:resolutionsystem`, `def:automatizable`, `def:polytimecheckable`, `def:feasibleinterp`, `def:interpolant`, `def:pigeonholefamily`, `cor:pigeonholespeedup`, `cor:nogbpoly`, and the Cook–Reckhow p-simulation of `def:psim`. The material of this Part belongs beside those rather than beside the SIC definitions, and the definitional floor is the one new object it contributes.

15.1 The unit problem in $\|\cdot\|$

v45 defines two proof measures: $\|\varphi\|_F(\Gamma)$, the line count, and $\|\varphi\|_F^{\mathrm{size}}(\Gamma)$, the sum of formula lengths. It observes that $\mathrm{size}(\pi) \geq |\pi|$ and that the canonical machines, the Proof-Horizon Theorem and the transaction-count apparatus all relativise verbatim under $|\pi| \mapsto \mathrm{size}(\pi)$.

The observation offered here concerns a *third* measure that v45 does not have. Define the *logical-line count* $\|\varphi\|_F^{\log}(\Gamma)$ as $|\pi|$ restricted to steps whose conclusion begins `|-`. Over a Metamath corpus this is the quantity a prover's difficulty actually tracks, and it is far smaller than either of v45's measures:

- $|\pi|$ counts notation and inference together;
- $\text{size}(\pi)$ is worse still, since syntax steps push large formulas onto the stack;
- $\|\cdot\|^{\log}$ counts only inference.

The measurement, and a trap in reporting it. On a random sample of 6,000 `set.mm` proofs decompressed by the CV:

	Raw steps	Logical steps
Median	173	12
Mean	1,678	79.1
Ratio of medians		14.4
Ratio of aggregates (total raw / total logical)		21.2
Share of all executed steps that are formula-building		95.3%

These are three different numbers and they answer three different questions. 14.4 is the correction factor for a *typical* proof; 21.2 is the correction factor for *total work* across the corpus; 95.3% is the fraction of steps a two-phase prover generates rather than searches for. The distributions are heavily right-skewed — mean raw length 1,678 against median 173 — which is why the aggregate and median ratios differ by 47%. An earlier revision of this document quoted a median raw length alongside the aggregate ratio as though one followed from the other; they do not.

$\|\cdot\|^{\log}$ relativises exactly as the size measure does, by the same concatenation argument v45 uses, so no new theory is required. What is required is that every numerical claim about $\|\cdot\|$ over a Metamath corpus say which of the three measures is meant, and whether a stated ratio is by medians or in aggregate. Predator's measurements use $\|\cdot\|^{\log}$.

16 Conclusion

16.1 What is proved

Three theorems, each conditional on nothing beyond v45's definitions. Theorem 9.1 shows the cross-branch reuse rate is a property of the search policy: the canonical machine makes every lemma available to both branches, so $\kappa^{\text{av}} = 1$ uniformly in the conjecture, and any other value is manufactured by targeting. Theorem 10.1 identifies cross-branch insertion with v45's derived-rule construction, giving a settlement dividend $\Delta(c) \geq 0$ and the corollary that a branch which never halts is not thereby wasted. Theorem 11.2 shows a chain deficit produces an unbounded policy-relative speedup, and Corollary 11.4 explains why a layer of independent lemmas cannot: the links must be premises of one another.

Alongside these, Proposition 6.2 records that clocked simulation does not collapse, which reverses Revision 1's verdict on the settlement-region programme, and Proposition 8.2 verifies that the settlement machine is a SIC in v45's sense.

16.2 What is measured

The certificate verifier reproduces the accepted status of all 47,572 `set.mm` proofs, and the four Predator_7.1 certificates verify in full corpus context. Against that, Section 5 reports that all four are one-step citations of pre-existing duplicates, and that 16.8% of the corpus's logical assertions are vulnerable to the same effect. The corpus statistics — the syntax-to-logic ratios, the dependency-closure calibration, the absence of any machine model — are all recomputable from the commands stated where they appear.

16.3 What is conjectured, and what is merely hoped

Conjecture 11.5 asserts that chain deficits exist over the condensed-detachment fragment. Problem 12.1 asks for a number that does not currently exist. Definition 5.2 names a quantity — non-trivial reach — whose value for Predator_7.1 is, on present evidence, plausibly zero.

16.4 The unifying observation

The two open problems in this document have the same shape, and noticing that is the most useful thing in it.

> Theorem 11.2 gives a policy-relative separation. Upgrading it to v45's Abstract Asymptotic Speedup Principle requires condition (c′) of Remark 11.3: *every* F-proof of c_n has length at least $g(n) + h(n)$. Part III's Section 15 records that no technique is known for establishing lower bounds of that form in a strong system — not even for a single tautology in Frege.
>
> **So the obstacle to finishing Part II is the obstacle to finishing Part III.** Both need a proof-length lower bound. The speedup principle is not blocked by a lack of ingenuity in constructing families; it is blocked by proof complexity, and it will stay blocked until proof complexity moves.

This has a constructive consequence and a deflationary one. Constructively, on a fragment small enough for breadth-first search to terminate, $\|c_n\|$ is *computable* rather than merely bounded, so a candidate family can be verified outright for small n — which is exactly what the Predator_5 machinery does and why that fragment remains useful despite being a toy. Deflationarily, no

amount of work on the search side of this program will settle a Clay problem, and the framework's own Proof-Horizon Theorem is what says so.

16.5 Next steps, in order of tractability

1. **Measure κ^{us}** (Problem 12.1). Requires the shared index and origin logging; no new theory.
2. **Measure non-trivial reach** (Definition 5.2). Requires the admissible-index filter, which is a substitution check against a prefix of the corpus.
3. **Restore the Predator_4 ranker** to Predator_7.1, which currently runs with `rank=None`.
4. **Search for a chain deficit** (Conjecture 11.5) on the condensed-detachment fragment, verifying (c') by exhaustive search for small n.
5. **Re-express the measurements in logical-line units** (Section 15.1), applying the correction factor appropriate to each quantity — 14.4 where a typical proof is meant, 21.2 where total work is meant.

17 A Collection of Objects Discussed Herein

These tables follow the format of the corresponding tables in v45. The **First defined** column names the originating document: **v45** means *Depths of a Simulation* version 45, with its section or label; **here** means this document, with its numbered environment. Objects marked **here** are the only ones for which this document claims priority.

Table 1: Machines and search objects

Name of object	Object type	First defined	Role
$E_{F,Y}$	SIC; enumerating ATP	v45, §2 (`def:canonicalenum`)	Canonical brute-force benchmark; $H \equiv 0$, never halts
$E_{F,Y,\varphi}$	SIC; target-search ATP	v45, §2 (`def:canonicaltarget`)	Canonical target benchmark
$B_{F,Y}$	SIC; breadth-first ATP	v45, §2	Shortest-proof horizon machine
$B_{F,Y,\varphi}$	SIC; breadth-first target-search ATP	v45, §2	The machine realising $\tau = \|\varphi\|$
F_T	Configuration formal system	v45, App. (`def:configsys`)	One computation step as one inference
E_T, M_T	Trace SICs	v45, App. (`def:enumtrace`, `def:halttrace`)	Turing machine as SIC; enumerating and halting
M_K	Non-computable SIC	v45, App. (`ex:beyondturing`)	Witness that SICs exceed Turing machines
$M^{\otimes}$	**Settlement machine**	**here**, Def. 8.1	Two branches, shared store, alternating stages
M^{sep}	Separated settlement machine	**here**, Thm. 10.1	Control: same branches, no shared store
Λ_m	**Lemma store**	**here**, Def. 8.1	Ground lemmas closed by either branch

Table 2: Simulation relations and search policies

Name of object	Object type	First defined	Role
$S_M \times S_N$	Weak simulation witness	v45, §3 (`thm:weakcollapse`)	Witness for universal weak simulation; the collapse
id_{S_M}	Synchronous simulation map	v45, §3	Reflexive strong simulation
(f, ρ)	**Clocked simulation pair**	**v45, §3**	**The principal relation**; f injective, ρ an admissible clock
$(g \circ f, \sigma \circ \rho)$	Clocked simulation pair	v45, §3	Transitivity construction
ρ	Admissible clock	v45, §3	Nondecreasing, unbounded, $\rho(1) = 1$
$\preceq_{\mathrm{clk}}$, written $\preceq$	Preorder on SICs	v45, §3	Does *not* collapse; see Prop. 6.2
$\preceq_{\mathrm{p}}$	p-simulation preorder	v45, §10 (`def:psim`)	Cook–Reckhow; tracks size, not stages
t, f_t	Recoding and its witness	v45, §4	Invertible coding isomorphism
Σ	Nondeterministic proof-search policy	v45, §5	Branch generator / heuristic
Proof-covering	Property of a policy	v45, §5	Every finite proof shadowed by a branch

Table 3: Metrics for automated theorem proving

Name of object	Object type	First defined	Role
$\|\varphi\|_F(\Gamma)$	Proof-length measure	v45, §2	Line count of a shortest proof; the proof horizon
$\|\varphi\|_F^{\text{size}}(\Gamma)$	Size-relativised measure	v45, §10 (`def:sizefunction`)	Sum of formula lengths; apparatus relativises
$\|\varphi\|_F^{\log}(\Gamma)$	**Logical-line count**	**here**, §15.1	$\|\pi\|$ restricted to `|-` steps; 14.4× smaller by medians, 21.2× in aggregate
$L_F(\Gamma, m)$	Proof layer	v45, §2	Exact shortest-proof stratum
$\tau_M(\Gamma, \varphi)$	Search-cost measure	v45, §7	Target transaction count; counts stage advances
$\sigma_M(\Gamma, c)$	**Settlement count**	**here**, Def. 8.4	Stages until c or $\neg c$ appears; extends τ to two-sided targets
μ	Materialisation count	**here** (cf. v45 §7 remark)	Distinct formulas built before halting; policy-determined
κ^{av}, κ^{us}	**Cross-branch reuse rates**	**here**, Def. 8.6	Fraction of lemmas crossing the branch divide, by availability and by use
$\Delta(c)$	**Settlement dividend**	**here**, Thm. 10.1	Stages saved by sharing the store
Non-trivial reach	**Evaluation metric**	**here**, Def. 5.2	Targets proved under the admissible index
$\sigma > \omega$	**Break-even rule**	**here**, App. A	Node speedup against per-node overhead
$P_F(\Gamma, \varphi, n)$	Counting function	v45, §8	Number of length-n proofs
$s_F(\Gamma)$	Proof-complexity function	v45, §10	Size of a shortest refutation of $\bot$
$T(\pi)$	**Verification transcript size**	**here**, Def. 13.1	Symbols materialised on the stack; the right unit for NP membership
$D_\Sigma(\varphi)$	**Definitional floor**	**here**, Def. 14.2	Assertions needed before φ is expressible

Table 4: Formal systems, rules, and results invoked

Name of object	Object type	First defined	Role
$\mathrm{Con}_F(\Gamma)$	Consequence operator	v45, §1	Least inference-closed extension
D_F	One-step closure operator	v45, §1	Direct-consequence expansion
F^D	Macro-augmented system	v45, §7	F plus derived rules; conservative
d_λ	**Nullary derived rule**	**here**, Def. 8.7	Cross-branch insertion of a closed lemma
Proof-Horizon Thm.	Theorem	v45 (`thm:proofhorizon`)	First halting stage of $B_{F,Y,\varphi}$ is $\|\varphi\|$
Branch-Covering Thm.	Theorem	v45 (`thm:branchcovering`)	Proof-covering policies each φ by $\|\varphi\|$
Branch Soundness Thm.	Theorem	v45 (`thm:branchsoundness`)	$p_m \subseteq \mathrm{Con}_F(\Gamma) \cap Y$
Conservative Compilation Thm.	Theorem	v45 (`thm:compilation`)	Finite workloads compile to $\tau \leq 2$
Abstract Asymptotic Speedup	Theorem	v45 (`thm:asymptspeedup`)	Needs $q(n)$ bounding *all* F-proofs
Chain deficit	**Hypothesis on a family**	**here**, Def. 11.1	Sufficient for policy-relative speedup
Admissible index	**Evaluation protocol**	**here**, Def. 5.1	Excludes duplicate statements, not merely labels
$\textsc{Short-Proof}_\Sigma$	**Language in** NP	**here**, Prop. 13.4	Bounded provability, certified by the CV

Appendices

A History of Predator

Each version exists because the previous one hit a specific wall. The walls are more informative than the versions.

Predator_1 — forward search, propositional (`predator.py`, 876 lines)

A trainable prover over a propositional fragment. Forward search: apply rules, generate consequences; brute force means adjoining every direct consequence, and effort is measured in expansions. Feasible only because the fragment is small enough to enumerate. **Wall:** the fragment is a toy.

Predator_2 — premise selection over ZFC (`predator2.py`, 960 lines)

Re-deriving a `set.mm` proof needs the full substitution machinery, so forward search does not transfer. But `set.mm` hands you the premise structure for free: every proof names what it cites. Predator_2 therefore asks the question a working prover faces — given a goal, which of tens of thousands of statements does its proof use? — scoring with recall@k and *effort*, the rank of the last true premise. **Wall:** ranks, cannot prove.

Predator_3 — premise selection at scale (`predator3.py`, 1,128 lines)

The same task with corpus handling made to scale. **Wall:** unchanged.

Predator_4 — learning to rank (`predator4.py`, 1,226 lines)

A learned ranker trained on 903,356 logical references extracted from all of `set.mm`. The strongest component of the line and the one still worth reviving. **Wall:** still no search. A ranking is not a proof.

Predator_5 — a learned search policy (`predator5.py`, 794 lines)

The pivot. Three changes, each load-bearing:

1. **The negatives change.** A policy's negatives must be the other edges applicable *at that state*, not lemmas sampled from elsewhere in the library. Ranking an edge against a lemma from a different part of the search answers a question no search ever asks.

2. **The labels are computed, not read.** A `set.mm` proof is an upper bound on the shortest proof, not the shortest proof. Predator_5 runs breadth-first search first, which returns the true distance, and marks every edge on some shortest path. The price is that the fragment must be small enough for BFS to finish — which, per Remark 11.3, is also what makes that fragment the only place a chain deficit can currently be verified.

3. **There is a completeness theorem to lose.** The Branch-Covering Theorem requires Σ to be proof-covering. Reordering Σ preserves that; truncating it does not. So `--mode reorder` keeps the theorem and `--mode prune` surrenders it.

Measured: every discarding strategy lost to unguided BFS on both time and solve rate. Beam search expanded *more* nodes than BFS (114.5 vs 90.0), solved 70% against 100%, and ran 6.6× slower. **Wall:** a 287-state fragment, not `set.mm`.

Predator_6 — expert iteration (`predator6.py`, 772 lines)

Train on BFS-certified geodesics, use the policy to reach targets beyond the BFS horizon, retrain. **Measured:** no improvement on the fragment — and this was *predicted*, because BFS already prices every target there, so the certified labels are already optimal. A hypothesis of mine about the expert-iteration frontier was refuted by a controlled sweep and withdrawn. **Wall:** the fragment cannot show what expert iteration is for.

Predator_7 — ranking coupled to search (`predator7.py`, 726 lines)

Couples Predator_4's ranker to Predator_5's search over the full corpus. The missing piece had always been a parser: a `set.mm` statement is a flat token sequence with nothing marking which tokens group. The grammar is the 1,441 syntax axioms already in the file. **Wall:** matching confines it to determined assertions; `ax-mp` and `syl` are not determined.

Predator_7.1 — unification and metavariables (`predator71.py`, 689 lines)

Replaces matching with unification, lifting the determinedness ceiling. Emits certificates; the CV checks them. **Wall:** the ranker was dropped in the rewrite and `prove` still runs with `rank=None`; and, per Section 5, its demonstrated results are duplicate lookups.

Supporting programs

File	Lines	Role
`metamath_cv.py`	916	the certificate verifier
`setmm_grammar.py`	578	grammar extraction, parsing, round-trip test
`tau.py`	706	τ, μ, derived rules, family generator
`arena.py`	320	five strategies timed, break-even test

The break-even rule

Proof-covering says which policies are *sound*; nothing in v45 says which are *worth running*. Let $\sigma_{\text{be}} = E_{\text{BFS}}/E_\pi$ be the node speedup and $\omega = c_\pi/c_{\text{BFS}}$ the per-node overhead. Then π is faster in real time iff $\sigma_{\text{be}} > \omega$. Measured: Predator_1 had $\sigma_{\text{be}} = 18.1$, $\omega = 41$, so 2.5× *slower*; Predator_5 had $\sigma_{\text{be}} = 2.14$, $\omega = 1.2$, so 1.76× faster. The smaller node advantage won, because a precomputed graph makes nodes cheap — but a precomputed graph means the proofs are already known.

B The four certificates in full

```
$( Predator_7.1 proof of axin1 $)
$[ set.mm $]
chk $p |- ( ( ph -> -. ph ) -> -. ph ) $= wph pm2.01 $.

$( Predator_7.1 proof of falanfal $)
```

```
$[ set.mm $]
chk $p |- ( ( F. /\ F. ) <-> F. ) $= wfal anidm $.

$( Predator_7.1 proof of tbw-ax4 $)
$[ set.mm $]
chk $p |- ( F. -> ph ) $= wph falim $.

$( Predator_7.1 proof of axia1 $)
$[ set.mm $]
chk $p |- ( ( ph /\ ps ) -> ph ) $= wph wps simpl $.
```

Each verifies. Each is a single citation. See Section 5.

C Source code

Complete current sources for the four files the results depend on. They are included by `\VerbatimInput` from the working folder, so recompiling this document requires the `.py` files to sit beside the `.tex`.

Not included, and why. The historical provers `predator.py` (876), `predator2.py` (960), `predator3.py` (1,128), `predator4.py` (1,226) and `predator6.py` (772) are present in the working folder but not listed; at roughly 5,000 further lines they would triple this document without supporting any claim in it. Predator_4 is the exception worth noting: it is the ranker the conclusion recommends restoring.

Files I do not have. No trained model weights are included; the Predator_4 ranker's fitted parameters were not preserved across the rewrite and would have to be retrained. **The settlement machine of Part II is not implemented** — Part II is theory and a measurement protocol, and no code for it exists yet.

C.1 `metamath_cv.py` — the certificate verifier

```
#!/usr/bin/env python3
"""
metamath.py -- a Metamath parser and CERTIFICATE VERIFIER (CV).

Brian Tenneson.  One file, no dependencies.

THIS IS A CV, NOT A PROOF ASSISTANT
-----------------------------------
A proof assistant is interactive: tactics, goal state, a human steering the
construction.  Nothing here does that.  You hand this program a proof
CERTIFICATE -- the RPN label sequence after $= -- and it returns a verdict.
It offers no help; it only judges.

That division is the point.  The searcher may be large, heuristic, machine
learned and wrong; the CV is a few hundred lines implementing one rule, and
nothing the searcher claims counts until the CV agrees.  A system with that
shape satisfies the DE BRUIJN CRITERION: every proof, however found, is
checkable by a kernel small enough to audit by hand.

    Predator_7.1   ATP.  Searches, emits certificates.  Untrusted.
    metamath.py    CV.   Checks certificates.  Trusted, and small.

    python metamath.py verify demo0.mm          verify every proof in a file
    python metamath.py stats   set.mm           corpus statistics
    python metamath.py show    set.mm sbth      one theorem, in full

WHY A CV AND NOT JUST A PARSER
------------------------------
The earlier setmm_parser.py did not verify anything, and it had a bug that only
a verifier would have caught: in Metamath the LABEL PRECEDES THE KEYWORD,

    th1 $p |- t = t $= ...proof... $.
    ^^^ label            ^^ keyword

so a parser that skips '$p' and reads the next token gets '|-' as the name of
```

```
every theorem in the file.  It produces a plausible-looking JSON of 43,000
entries all named '|-'.  Nothing downstream can detect this.  A CV can:
a proof that does not check is a proof that was not read correctly.

This matters for the central question of the project.  "w is in stage m of M"
means w is a theorem -- but only if the steps really are instances of the rules.
The CV is what makes stage-membership a proof rather than an assertion:
thm:branchsoundness says p_m is contained in Con_F(Gamma), and the CV turns
that from a claim about the program into a checkable fact.

WHAT METAMATH ACTUALLY IS
-------------------------
Metamath has ONE rule: substitution of symbol sequences for variables.  There is
no built-in logic, no unification, no types beyond what set.mm declares.  An
assertion is verified by an RPN stack machine:

    for each label in the proof:
        if it names a hypothesis, PUSH its statement
        if it names an assertion, POP its mandatory hypotheses, solve for the
            substitution they force, check the $e hypotheses match under that
            substitution, check the disjoint-variable conditions, and PUSH the
            conclusion under that substitution
    at the end the stack must hold exactly the statement being proved

The substitution is on VARIABLES -> SYMBOL SEQUENCES.  That is the whole system.
Its smallness is why a verifier is a few hundred lines and why set.mm can be
trusted at all.

MANDATORY HYPOTHESES (the part that is easy to get wrong)
---------------------------------------------------------
An assertion's frame is not "the hypotheses in scope".  It is:

  * the $e hypotheses in scope, in declaration order, and
  * the $f hypotheses, in declaration order, for exactly those variables that
    occur in the assertion or in one of those $e hypotheses,
  * the $d disjointness pairs in scope restricted to those variables.

$f hypotheses for variables that do NOT occur are not mandatory and must not be
popped.  Getting this wrong shifts the whole stack and every proof fails.

COMPRESSED PROOFS
-----------------
set.mm stores proofs compressed:

    $= ( label1 label2 ... ) ABCZDEF $.

The letters are base-20/base-5 numbers: A-T are final digits (1..20), U-Y are
continuation digits (1..5), and Z marks the previous step for reuse.  A decoded
number indexes, in order, the mandatory hypotheses, then the parenthesised
labels, then the Z-marked backreferences.  Uncompressed proofs are also
supported.  Proofs containing '?' are incomplete and are reported as such
rather than counted as verified.
"""
from __future__ import annotations
import argparse, itertools, os, sys, time
from collections import defaultdict

VERSION = "1.1"          # 1.1 = processes $[ file $] includes

WORKDIR = r"C:\google drive\Automated Theorem Proving"
if os.path.isdir(WORKDIR):
    os.chdir(WORKDIR)


class MMError(Exception):
    pass


# ============================================================================
#  tokenizer:  whitespace-separated, $( ... $) comments, $[ file $] includes
# ============================================================================
class Toks:
    def __init__(self, text):
        self.toks = text.split()
        self.i = 0

    def next(self):
        """Next token, with comments stripped."""
        while self.i < len(self.toks):
            t = self.toks[self.i]; self.i += 1
            if t == "$(":
                # comments do not nest; scan to the closing $)
                while self.i < len(self.toks) and self.toks[self.i] != "$)":
                    self.i += 1
                self.i += 1
                continue
            return t
        return None

    def readuntil(self, end):
        out = []
        while True:
```

```
            t = self.next()
            if t is None:
                raise MMError("unterminated statement, expected %s" % end)
            if t == end:
                return out
            out.append(t)


# ====================================================================
#  scope frames
# ====================================================================
class Frame:
    __slots__ = ("v", "d", "f", "e")
    def __init__(self):
        self.v = set()      # active variables
        self.d = set()      # disjoint pairs, stored sorted
        self.f = []         # [(label, typecode, var)] in declaration order
        self.e = []         # [(label, statement)]     in declaration order


class FrameStack(list):
    def push(self):
        self.append(Frame())

    def pop_frame(self):
        self.pop()

    def add_v(self, v):
        self[-1].v.add(v)

    def add_f(self, label, typecode, var):
        if not self.lookup_v(var):
            raise MMError("$f for inactive variable %s" % var)
        self[-1].f.append((label, typecode, var))

    def add_e(self, label, stat):
        self[-1].e.append((label, stat))

    def add_d(self, varlist):
        self[-1].d.update((min(x, y), max(x, y))
                          for x, y in itertools.product(varlist, varlist)
                          if x != y)

    def lookup_v(self, tok):
        return any(tok in fr.v for fr in self)

    def lookup_f(self, var):
        for fr in reversed(self):
            for label, tc, v in fr.f:
                if v == var:
                    return tc
        return None

    def all_dvs(self):
        """EVERY disjoint pair active here, not just those on mandatory vars.

        A theorem's own proof may introduce DUMMY variables -- ones that occur
        nowhere in its statement.  equid proves  |- x = x  using an auxiliary z.
        The DV conditions governing those dummies are active in the scope but
        are filtered out of the mandatory frame, so verifying equid's own proof
        needs this unrestricted set.  Using the mandatory set here rejects
        equid, ax7, exgen, spnfw and spsv -- correct proofs, wrong reader."""
        return {(x, y) for fr in self for (x, y) in fr.d}

    def make_assertion(self, stat):
        """Compute the frame of an assertion: (dvs, f_hyps, e_hyps, stat).

        The dvs here are restricted to mandatory variables, which is what a
        LATER proof must satisfy when it applies this assertion as a step.  It
        is deliberately NOT the set used to check this assertion's own proof --
        see all_dvs above.

        The other subtle part is which $f hypotheses are mandatory: exactly
        those whose variable occurs in the assertion or in an active $e, taken
        in declaration order.  A $f for an unused variable is NOT mandatory."""
        e_hyps = [(lab, s) for fr in self for (lab, s) in fr.e]
        mand_vars = {tok for _, hyp in e_hyps for tok in hyp
                     if self.lookup_v(tok)}
        mand_vars |= {tok for tok in stat if self.lookup_v(tok)}

        dvs = {(x, y) for fr in self for (x, y) in fr.d
               if x in mand_vars and y in mand_vars}

        f_hyps = []
        seen = set()
        for fr in self:                     # outermost first = declaration order
            for label, tc, v in fr.f:
                if v in mand_vars and v not in seen:
                    f_hyps.append((label, tc, v))
                    seen.add(v)
        return (dvs, f_hyps, e_hyps, stat)

```

```
220
221 # ============================================================================
222 #  substitution:  variable -> symbol sequence.  This is the entire rule.
223 # ============================================================================
224 def apply_subst(stat, subst):
225     out = []
226     for tok in stat:
227         s = subst.get(tok)
228         if s is None:
229             out.append(tok)
230         else:
231             out.extend(s)
232     return out
233
234
235 # ============================================================================
236 #  the database
237 # ============================================================================
238 class MM:
239     def __init__(self):
240         self.constants = set()
241         self.variables = set()    # every variable ever declared, for DV checks
242         self.fs = FrameStack()
243         self.labels = {}          # label -> ('$a'|'$p'|'$f'|'$e', data)
244         self.order = []           # labels in file order
245         self.proofs = {}          # label -> raw proof token list
246         self.scope_dvs = {}       # $p label -> unrestricted DVs at its scope
247         self.included = set()     # files already pulled in by $[ ... $]
248         self.stats = defaultdict(int)
249
250     # ------------------------------------------------------------------ read
251     def read(self, toks, top=True):
252         if top:
253             self.fs.push()
254         label = None
255         while True:
256             tok = toks.next()
257             if tok is None:
258                 break
259
260             if tok == "$c":
261                 for c in toks.readuntil("$."):
262                     self.constants.add(c)
263
264             elif tok == "$v":
265                 for v in toks.readuntil("$."):
266                     self.fs.add_v(v)
267                     self.variables.add(v)
268
269             elif tok == "$f":
270                 stat = toks.readuntil("$.")
271                 if label is None:
272                     raise MMError("$f with no label")
273                 if len(stat) != 2:
274                     raise MMError("malformed $f %s" % label)
275                 self.fs.add_f(label, stat[0], stat[1])
276                 self.labels[label] = ("$f", [stat[0], stat[1]])
277                 self.order.append(label)
278                 label = None
279
280             elif tok == "$e":
281                 stat = toks.readuntil("$.")
282                 if label is None:
283                     raise MMError("$e with no label")
284                 self.fs.add_e(label, stat)
285                 self.labels[label] = ("$e", stat)
286                 self.order.append(label)
287                 label = None
288
289             elif tok == "$a":
290                 stat = toks.readuntil("$.")
291                 if label is None:
292                     raise MMError("$a with no label")
293                 self.labels[label] = ("$a", self.fs.make_assertion(stat))
294                 self.order.append(label)
295                 self.stats["axioms"] += 1
296                 label = None
297
298             elif tok == "$p":
299                 rest = toks.readuntil("$.")
300                 if label is None:
301                     raise MMError("$p with no label")
302                 if "$=" not in rest:
303                     raise MMError("$p %s has no $=" % label)
304                 cut = rest.index("$=")
305                 stat, proof = rest[:cut], rest[cut + 1:]
306                 self.labels[label] = ("$p", self.fs.make_assertion(stat))
307                 self.proofs[label] = proof
308                 # captured at declaration point, while the scope is still open
309                 self.scope_dvs[label] = self.fs.all_dvs()
310                 self.order.append(label)
311                 self.stats["theorems"] += 1
```

```
                label = None

            elif tok == "$d":
                self.fs.add_d(toks.readuntil("$."))

            elif tok == "${":
                self.fs.push()

            elif tok == "$}":
                self.fs.pop_frame()

            elif tok == "$[":
                # $[ file $] splices another database in at this point.  It was
                # previously discarded on the grounds that set.mm is one file --
                # but a prover that emits a proof of a set.mm theorem must write
                # a file that REFERS to set.mm, and without includes such a file
                # cites labels it never declares.  That is why axin1_p71.mm
                # failed with "unknown label wph": the proof was fine, the
                # container was not.
                fn = " ".join(toks.readuntil("$]"))
                if fn not in self.included:
                    self.included.add(fn)
                    if not os.path.exists(fn):
                        raise MMError("include not found: %s" % fn)
                    with open(fn, encoding="utf-8", errors="replace") as f:
                        self.read(Toks(f.read()), top=False)

            elif tok.startswith("$"):
                raise MMError("unknown command %s" % tok)

            else:
                # THE LABEL COMES FIRST.  This is the line the old parser got
                # wrong, and the reason every theorem came out named '|-'.
                label = tok

    # ------------------------------------------------ decompress a proof
    def decompress(self, label, proof):
        """Expand a compressed proof into a flat list of labels."""
        dvs, f_hyps, e_hyps, stat = self.labels[label][1]
        mand = [l for l, _, _ in f_hyps] + [l for l, _ in e_hyps]

        if proof[0] != "(":
            return proof                        # already uncompressed
        end = proof.index(")")
        extra = proof[1:end]
        letters = "".join(proof[end + 1:])

        pool = mand + extra                     # 1-based indexing below
        out, saved, n = [], [], 0
        for ch in letters:
            if ch == "?":
                out.append("?")
                n = 0
            elif "U" <= ch <= "Y":
                n = n * 5 + (ord(ch) - ord("U") + 1)
            elif "A" <= ch <= "T":
                n = n * 20 + (ord(ch) - ord("A") + 1)
                if n <= len(pool):
                    out.append(pool[n - 1])
                else:
                    k = n - len(pool) - 1
                    if k >= len(saved):
                        raise MMError("bad backreference %d in %s (only %d "
                                      "saved)" % (k + 1, label, len(saved)))
                    out.extend(saved[k])
                n = 0
            elif ch == "Z":
                # Z saves the subproof ending at the step just emitted.  The
                # backreference list grows ONLY here -- an earlier version
                # appended on every step, which made backreference 1 the first
                # single token instead of the marked subproof.  The Z test in
                # mmtest/ztest.mm exists to catch exactly that.
                saved.append(self._subproof(out))
            else:
                raise MMError("bad character %r in compressed proof %s"
                              % (ch, label))
        return out

    def _subproof(self, out):
        """The trailing segment of `out` that forms one complete subproof."""
        need, i = 1, len(out)
        while need and i:
            i -= 1
            lab = out[i]
            if lab == "?":
                need -= 1
                continue
            typ, data = self.labels[lab]
            if typ in ("$e", "$f"):
                need -= 1
            else:
                dvs, f_hyps, e_hyps, _ = data
```

```
                need += len(f_hyps) + len(e_hyps) - 1
        return out[i:]

    # ------------------------------------------------------------ verify
    def verify(self, label):
        """Verify one $p.  Returns 'ok', 'incomplete', or raises MMError."""
        _mand_dvs, f_hyps, e_hyps, conclusion = self.labels[label][1]
        # check this proof against the UNRESTRICTED scope DVs, so that dummy
        # variables introduced inside the proof are covered
        dvs = self.scope_dvs.get(label, _mand_dvs)
        proof = self.decompress(label, self.proofs[label])
        if "?" in proof:
            return "incomplete"

        stack = []
        for step in proof:
            if step not in self.labels:
                raise MMError("%s: unknown label %s" % (label, step))
            typ, data = self.labels[step]

            if typ in ("$e", "$f"):
                stack.append(list(data))
                continue

            s_dvs, s_f, s_e, s_concl = data
            npop = len(s_f) + len(s_e)
            if npop > len(stack):
                raise MMError("%s: stack underflow at %s" % (label, step))
            base = len(stack) - npop

            # the $f hypotheses determine the substitution
            subst = {}
            for j, (_, tc, var) in enumerate(s_f):
                entry = stack[base + j]
                if entry[0] != tc:
                    raise MMError("%s: type mismatch at %s: %s vs %s"
                                  % (label, step, entry[0], tc))
                subst[var] = entry[1:]

            # the $e hypotheses must then match exactly
            for j, (_, e_stat) in enumerate(s_e):
                entry = stack[base + len(s_f) + j]
                if apply_subst(e_stat, subst) != entry:
                    raise MMError("%s: hypothesis mismatch at %s" % (label, step))

            # disjoint-variable conditions.  Membership is tested against the
            # GLOBAL variable set: by verification time the scope stack has
            # been popped back to the outermost frame, so fs.lookup_v would
            # miss any variable declared inside a ${ ... $} block.
            for x, y in s_dvs:
                sx = [t for t in subst.get(x, ()) if t in self.variables]
                sy = [t for t in subst.get(y, ()) if t in self.variables]
                for a, b in itertools.product(sx, sy):
                    if a == b or (min(a, b), max(a, b)) not in dvs:
                        raise MMError("%s: disjoint-variable violation "
                                      "(%s,%s) -> (%s,%s) at %s"
                                      % (label, x, y, a, b, step))

            del stack[base:]
            stack.append(apply_subst(s_concl, subst))

        if len(stack) != 1:
            raise MMError("%s: proof ends with %d entries on the stack"
                          % (label, len(stack)))
        if stack[0] != conclusion:
            raise MMError("%s: proved the wrong statement\n  got      %s\n"
                          "  expected %s" % (label, " ".join(stack[0]),
                                             " ".join(conclusion)))
        return "ok"


# ====================================================================
#  loading
# ====================================================================
def load(path, say=print):
    if not os.path.exists(path):
        raise SystemExit(
            "%s not found in %s\n"
            "  get set.mm with:\n"
            "    curl -o set.mm https://raw.githubusercontent.com/"
            "metamath/set.mm/develop/set.mm" % (path, os.getcwd()))
    t0 = time.time()
    with open(path, encoding="utf-8", errors="replace") as f:
        text = f.read()
    say("  %s: %.1f MB" % (path, len(text) / 1e6))
    mm = MM()
    mm.read(Toks(text))
    say("  parsed in %.1fs: %s axioms, %s theorems, %s constants"
        % (time.time() - t0, f"{mm.stats['axioms']:,}",
           f"{mm.stats['theorems']:,}", f"{len(mm.constants):,}"))
    return mm

```

```
496
497 # ====================================================================
498 #  commands
499 # ====================================================================
500 def cmd_verify(a):
501     print("\n" + "=" * 74)
502     print("  metamath.py v%s  --  certificate verifier" % VERSION)
503     print("=" * 74 + "\n")
504     mm = load(a.file)
505
506     names = [l for l in mm.order if mm.labels[l][0] == "$p"]
507     if a.limit:
508         names = names[:a.limit]
509     if a.only:
510         names = [l for l in names if l in set(a.only)]
511
512     print("\n  verifying %s proofs..." % f"{len(names):,}")
513     t0 = time.time()
514     ok = inc = 0
515     failures = []
516     for i, label in enumerate(names, 1):
517         try:
518             r = mm.verify(label)
519             if r == "ok":
520                 ok += 1
521             else:
522                 inc += 1
523         except MMError as e:
524             failures.append(str(e))
525             if len(failures) <= a.show_failures:
526                 print("    FAIL %s" % e)
527         except RecursionError:
528             failures.append("%s: recursion limit" % label)
529         if a.progress and i % a.progress == 0:
530             print("    %s/%s  (%.0f/s)"
531                   % (f"{i:,}", f"{len(names):,}", i / max(time.time() - t0, 1e-9)))
532
533     dt = time.time() - t0
534     print("\n  " + "-" * 70)
535     print("  verified   %s" % f"{ok:,}")
536     print("  incomplete %s" % f"{inc:,}")
537     print("  FAILED     %s" % f"{len(failures):,}")
538     print("  elapsed    %.1fs  (%.0f proofs/s)" % (dt, len(names) / max(dt, 1e-9)))
539     print("  " + "-" * 70)
540     if not failures and ok:
541         print("\n  Every proof checked.  Each verified label is a theorem of the")
542         print("  system: the stack machine reduced its proof to exactly its")
543         print("  statement using only substitution instances of the rules.\n")
544     elif failures:
545         print("\n  %d failures -- the reader is wrong, or the file is.\n"
546               % len(failures))
547     return 0 if not failures else 1
548
549
550 def classify(mm):
551     """Split every label into 'syntax' or 'logic'.
552
553     The criterion is the TYPECODE -- the first token of the statement.  In
554     set.mm a $a whose statement begins with 'wff', 'class' or 'setvar' is a
555     grammar rule that builds a formula; only a $a beginning with '|-' asserts
556     anything.  So
557
558         wcel $a wff A e. B $.        <- syntax: how to WRITE "A e. B"
559         ax-mp $a |- ps $.            <- logic:  an actual rule of inference
560
561     $f hypotheses are variable typings and are syntax by the same token.  This
562     is the distinction that raw step counts hide."""
563     kind = {}
564     for lab, (typ, data) in mm.labels.items():
565         if typ == "$f":
566             kind[lab] = "syntax"
567         elif typ == "$e":
568             kind[lab] = "hyp"
569         else:
570             stat = data[3]
571             kind[lab] = "logic" if stat and stat[0] == "|-" else "syntax"
572     return kind
573
574
575 def _dist(xs, label, pad=""):
576     xs = sorted(xs)
577     if not xs:
578         return
579     print("%s  %-14s median %s, mean %.0f, max %s"
580           % (pad, label, f"{xs[len(xs)//2]:,}", sum(xs) / len(xs), f"{xs[-1]:,}"))
581
582
583 def cmd_stats(a):
584     print("\n" + "=" * 74)
585     print("  metamath.py v%s  --  corpus statistics" % VERSION)
586     print("=" * 74 + "\n")
587     mm = load(a.file)
```

```
    kind = classify(mm)

    thms = [l for l in mm.order if mm.labels[l][0] == "$p"]
    axs = [l for l in mm.order if mm.labels[l][0] == "$a"]

    # ---- what the "axioms" actually are ------------------------------
    ax_syntax = [l for l in axs if kind[l] == "syntax"]
    ax_logic = [l for l in axs if kind[l] == "logic"]
    ax_true = [l for l in ax_logic if l.startswith("ax-")]
    ax_def = [l for l in ax_logic if l.startswith("df-")]
    ax_other = [l for l in ax_logic
                if not l.startswith("ax-") and not l.startswith("df-")]

    print("\n  $a statements   %s" % f"{len(axs):,}")
    print("    syntax rules  %-8s  grammar: how to write a formula"
          % f"{len(ax_syntax):,}")
    print("    definitions   %-8s  df-*" % f"{len(ax_def):,}")
    print("    TRUE AXIOMS   %-8s  ax-*" % f"{len(ax_true):,}")
    if ax_other:
        print("    other |-      %-8s  %s" % (f"{len(ax_other):,}",
                                              ", ".join(ax_other[:5])))
    print("\n    the logical content of ZFC is those %d ax-* statements."
          % len(ax_true))
    print("    %s of the %s are grammar, not mathematics."
          % (f"{len(ax_syntax):,}", f"{len(axs):,}"))
    if ax_true:
        print("    %s" % ", ".join(sorted(ax_true)[:14]))

    if not a.logical:
        print("\n  theorems        %s" % f"{len(thms):,}")
        print("\n  (run with --logical to separate reasoning steps from")
        print("   formula-building steps; it decompresses every proof)\n")
        return

    # ---- per-proof step accounting -----------------------------------
    print("\n  decompressing %s proofs..." % f"{len(thms):,}")
    t0 = time.time()
    total, logic_only = [], []
    cites_all, cites_logic = defaultdict(int), defaultdict(int)
    skipped = 0
    for i, l in enumerate(thms, 1):
        try:
            proof = mm.decompress(l, mm.proofs[l])
        except (MMError, RecursionError, ValueError):
            skipped += 1
            continue
        nl = sum(1 for s in proof if kind.get(s) == "logic")
        total.append(len(proof))
        logic_only.append(nl)
        for s in set(proof):                      # distinct labels per proof
            cites_all[s] += 1
            if kind.get(s) == "logic":
                cites_logic[s] += 1
        if i % 10000 == 0:
            print("    %s/%s" % (f"{i:,}", f"{len(thms):,}"))
    print("    done in %.1fs%s"
          % (time.time() - t0,
             ", %d skipped" % skipped if skipped else ""))

    print("\n  PROOF LENGTH")
    _dist(total, "all steps")
    _dist(logic_only, "logical only")
    st, sl = sum(total), sum(logic_only)
    if st:
        print("\n    %s of %s steps are formula-building: %.0f%%"
              % (f"{st - sl:,}", f"{st:,}", 100 * (st - sl) / st))
        print("    logical steps are %.1fx fewer than the raw count suggests."
              % (st / max(sl, 1)))
        print("\n    A geodesic measured in raw steps is measuring notation.")
        print("    The logical-only column is the length worth minimising.")

    longest = max(zip(total, thms))
    print("\n  longest proof   %s at %s steps"
          % (longest[1], f"{longest[0]:,}"))

    print("\n  MOST-CITED, ALL LABELS  (proofs citing it, of %s)"
          % f"{len(total):,}")
    for lab, n in sorted(cites_all.items(), key=lambda kv: -kv[1])[:10]:
        print("    %-12s %7s   %s" % (lab, f"{n:,}", kind.get(lab, "?")))

    print("\n  MOST-CITED LOGICAL LABELS  <- the real premise-selection prior")
    for lab, n in sorted(cites_logic.items(), key=lambda kv: -kv[1])[:15]:
        print("    %-12s %7s" % (lab, f"{n:,}"))
    print()


def cmd_show(a):
    mm = load(a.file)
    label = a.label
    if label not in mm.labels:
        near = [l for l in mm.labels if l.startswith(label[:4])][:12]
        print("\n  %s not found." % label)
```

```
        if near:
            print("  did you mean: %s" % ", ".join(near))
        return 1

    typ, data = mm.labels[label]
    print("\n" + "=" * 74)
    print("  %s   (%s)" % (label, typ))
    print("=" * 74)
    if typ in ("$e", "$f"):
        print("\n  %s\n" % " ".join(data))
        return 0

    dvs, f_hyps, e_hyps, stat = data
    print("\n  statement")
    print("    %s" % " ".join(stat))
    if f_hyps:
        print("\n  mandatory $f hypotheses (%d)" % len(f_hyps))
        for l, tc, v in f_hyps:
            print("    %-10s %s %s" % (l, tc, v))
    if e_hyps:
        print("\n  mandatory $e hypotheses (%d)" % len(e_hyps))
        for l, s in e_hyps:
            print("    %-10s %s" % (l, " ".join(s)))
    if dvs:
        print("\n  disjoint variables")
        print("    %s" % ", ".join("(%s,%s)" % p for p in sorted(dvs)))

    if typ == "$p":
        proof = mm.decompress(label, mm.proofs[label])
        print("\n  proof: %d steps" % len(proof))
        print("    %s" % " ".join(proof[:40]) + (" ..." if len(proof) > 40 else ""))
        t0 = time.time()
        try:
            r = mm.verify(label)
            print("\n  verification: %s  (%.3fs)" % (r.upper(), time.time() - t0))
            if r == "ok":
                print("  -> %s is a theorem: its proof reduces to exactly its"
                      % label)
                print("     statement under the substitution rule.")
        except MMError as e:
            print("\n  verification: FAILED\n    %s" % e)
    print()
    return 0


SELFTEST = r"""
$c 0 + = -> ( ) term wff |- $.
$v t r s P Q $.
tt $f term t $.   tr $f term r $.   ts $f term s $.
wp $f wff P $.    wq $f wff Q $.
tze $a term 0 $.
tpl $a term ( t + r ) $.
weq $a wff t = r $.
wim $a wff ( P -> Q ) $.
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
a2 $a |- ( t + 0 ) = t $.
${ min $e |- P $.  maj $e |- ( P -> Q ) $.  mp $a |- Q $. $}

th1 $p |- t = t $=
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
  tt weq tt tze tpl tt weq tt tt weq wim tt a2
  tt tze tpl tt tt a1 mp mp $.

th1c $p |- t = t $= ( tze tpl weq a2 wim a1 mp )
  ABCADAADAEABCADABCADAADFAEABCAAGHH $.

th1z $p |- t = t $= ( tze tpl weq a2 wim a1 mp )
  ABCADZAADAEIIAADFAEABCAAGHH $.

badstack $p |- t = t $=
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
  tt weq tt tze tpl tt weq tt tt weq wim tt a2
  tt tze tpl tt tt a1 mp tze $.

badconcl $p |- t = t $= tt a2 $.

incomp $p |- t = t $= ? $.

$( ---- disjoint-variable cases ---------------------------------------- $)
$v x y z $.
vx $f term x $.   vy $f term y $.   vz $f term z $.
${ $d x y $.
   axd $a |- x = y $. $}
${ $d x z $.
   hdz   $e |- x = z $.
   elimd $a |- x = x $. $}

$( dummyvar proves |- x = x, whose only variable is x, via a proof that
   introduces z.  z is a DUMMY: it occurs nowhere in the statement, so it is
   absent from the mandatory frame, so the (x,z) condition it needs is absent
   from the mandatory DV set.  Checking the proof against the mandatory set
   rejects this -- which is how equid, ax7, exgen, spnfw and spsv failed. $)
```

```
$( $d x z $.
   dummyvar $p |- x = x $= vx vz vx vz axd elimd $. $)

$( dvbad applies axd with x and y both instantiated to x, violating $d x y.
   No $d is in scope.  MUST fail, or the DV check is vacuous. $)
dvbad $p |- x = x $= vx vx axd $.
"""

EXPECTED = {
    "th1":      "ok",          # uncompressed
    "th1c":     "ok",          # same proof, compressed
    "th1z":     "ok",          # same proof, compressed with Z backreferences
    "badstack": "fail",        # corrupted: wrong final step
    "badconcl": "fail",        # well-formed but proves the wrong statement
    "incomp":   "incomplete",  # contains ?
    "dummyvar": "ok",          # proof introduces a variable not in the statement
    "dvbad":    "fail",        # genuine disjoint-variable violation
}


def cmd_selftest(a):
    """Run the verifier against cases with known answers.

    A verifier that accepts everything is worthless, so half of these must
    FAIL.  th1/th1c/th1z are three encodings of one proof and must decode to
    byte-identical step lists -- that is what catches compression bugs."""
    print("\n" + "=" * 74)
    print("  metamath.py v%s  --  selftest" % VERSION)
    print("=" * 74 + "\n")

    mm = MM()
    mm.read(Toks(SELFTEST))

    steps = {}
    bad = 0
    for label, want in EXPECTED.items():
        try:
            got = mm.verify(label)
            steps[label] = len(mm.decompress(label, mm.proofs[label]))
        except MMError as e:
            got = "fail"
            steps[label] = None
        ok = (got == want)
        bad += (not ok)
        print("  %-9s expect %-11s got %-11s %s"
              % (label, want, got, "ok" if ok else "<-- WRONG"))

    n = {steps[k] for k in ("th1", "th1c", "th1z")}
    same = (len(n) == 1)
    print("\n  three encodings of one proof decode to %s"
          % ("the same %d steps  ok" % n.pop() if same
             else "DIFFERENT lengths %s  <-- WRONG" % sorted(x for x in n if x)))
    bad += (not same)

    print("\n  " + ("all checks passed" if not bad
                    else "%d CHECK(S) FAILED" % bad) + "\n")
    return 0 if not bad else 1


def cmd_search(a):
    """Find labels whose statement contains all the given tokens.

    Exists because guessing set.mm label names is unreliable.  Before writing
    'apply sbth here', check that sbth says what you think it says."""
    mm = load(a.file)
    kind = classify(mm)
    want = a.tokens
    hits = []
    for lab in mm.order:
        typ, data = mm.labels[lab]
        if typ in ("$e", "$f"):
            if a.all_types:
                stat = data
            else:
                continue
        else:
            stat = data[3]
        if a.logical_only and kind.get(lab) != "logic":
            continue
        if a.prefix and not lab.startswith(a.prefix):
            continue
        s = " ".join(stat)
        if all(w in s for w in want):
            hits.append((lab, typ, s))

    print("\n  %s match%s%s"
          % (f"{len(hits):,}", "" if len(hits) == 1 else "es",
             " for %s" % " + ".join(repr(w) for w in want) if want else ""))
    if a.prefix:
        print("  (label prefix %r)" % a.prefix)
    print()
    for lab, typ, s in hits[:a.limit]:
```

```
        if len(s) > 92:
            s = s[:89] + "..."
        print("    %-14s %-3s %s" % (lab, typ, s))
    if len(hits) > a.limit:
        print("\n    ... %s more (raise --limit)" % f"{len(hits) - a.limit:,}")
    print()


def main():
    ap = argparse.ArgumentParser(prog="metamath", description=__doc__,
            formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd")
    sub.add_parser("selftest", help="verify the verifier (no files needed)")

    q = sub.add_parser("search", help="find labels by statement content")
    q.add_argument("tokens", nargs="*", help="all of these must appear")
    q.add_argument("--file", default="set.mm")
    q.add_argument("--prefix", default=None, help="label starts with this")
    q.add_argument("--limit", type=int, default=40)
    q.add_argument("--logical-only", action="store_true",
                   help="skip grammar rules")
    q.add_argument("--all-types", action="store_true",
                   help="include $e and $f too")

    v = sub.add_parser("verify", help="verify proofs")
    v.add_argument("file", nargs="?", default="set.mm")
    v.add_argument("--limit", type=int, default=0, help="first N theorems")
    v.add_argument("--only", nargs="*", default=None, help="specific labels")
    v.add_argument("--progress", type=int, default=2000)
    v.add_argument("--show-failures", type=int, default=10)

    s = sub.add_parser("stats", help="corpus statistics")
    s.add_argument("file", nargs="?", default="set.mm")
    s.add_argument("--logical", action="store_true",
                   help="separate reasoning steps from formula-building steps "
                        "(decompresses every proof; slower)")

    w = sub.add_parser("show", help="show one theorem in full")
    w.add_argument("file", nargs="?", default="set.mm")
    w.add_argument("label")

    a = ap.parse_args()
    if a.cmd == "selftest": sys.exit(cmd_selftest(a))
    elif a.cmd == "search": cmd_search(a)
    elif a.cmd == "verify":  sys.exit(cmd_verify(a))
    elif a.cmd == "stats": cmd_stats(a)
    elif a.cmd == "show":  sys.exit(cmd_show(a))
    else: ap.print_help()


if __name__ == "__main__":
    sys.setrecursionlimit(20000)
    main()
```

C.2 `setmm_grammar.py` — grammar extraction and parsing

```
#!/usr/bin/env python3
r"""
setmm_grammar.py -- parse set.mm statements into trees, and match them.

    python setmm_grammar.py selftest              no files needed
    python setmm_grammar.py roundtrip set.mm      parse everything, re-serialise
    python setmm_grammar.py tree set.mm canth     show one statement's parse
    python setmm_grammar.py match set.mm canth    what could conclude this goal

WHY THIS IS THE MISSING PIECE
-----------------------------
Predator_5 searches by computing the admissible one-step extensions of a state.
On the condensed-detachment fragment that is easy: unify two implicational
formulas.  On set.mm it is not, because a statement is a FLAT TOKEN SEQUENCE:

    |- ( ( A -<_ B /\ B -<_ A ) -> A ~~ B )

Nothing in that string says which tokens group with which.  Matching it against
another statement's conclusion requires knowing that it is an implication whose
antecedent is a conjunction -- that is, it requires a PARSE.

The grammar is already in the file.  Every syntax axiom is a production rule:

    wi  $a wff ( ph -> ps ) $.      <- "a wff may be ( wff -> wff )"
    wcel $a wff A e. B $.           <- "a wff may be class e. class"
    cvv $a class _V $.              <- "a class may be _V"

1,441 such rules in set.mm, against 1,559 that assert something.  Extracting
them and parsing with them turns the corpus from strings into trees, and only
then can a prover ask "which assertions could conclude this goal?"

This is also the two-phase split the corpus statistics pointed at.  95% of proof
STEPS are formula construction, and those steps are deterministic given the
formula -- they are exactly the parse tree, serialised.  A prover that builds
```

```
trees directly never searches for them.

THE ROUND-TRIP TEST
-------------------
Grammar extraction is easy to get subtly wrong, and a wrong grammar produces
plausible trees.  So the test is mechanical: parse every statement, serialise
the tree back to tokens, and require the result to equal the input exactly.
‘roundtrip‘ reports the failure count.  It must be zero.

AMBIGUITY
---------
set.mm’s syntax is designed to be unambiguous, but this parser does not assume
it.  ‘roundtrip --ambiguous‘ reports statements admitting more than one parse.
Any such statement is a place where "the" tree is not well defined and a matcher
built on it would be unsound.
"""
from __future__ import annotations
import argparse, os, sys, time
from collections import defaultdict

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
try:
    from metamath import MM, Toks, load, MMError
except ImportError:
    raise SystemExit("setmm_grammar.py needs metamath.py in the same folder")

WORKDIR = r"C:\google drive\Automated Theorem Proving"
if os.path.isdir(WORKDIR):
    os.chdir(WORKDIR)

# 30000 without a matching C stack is a segfault waiting to
# happen; _run_with_big_stack below supplies the stack.
sys.setrecursionlimit(8000)


# ============================================================================
#  trees
# ============================================================================
class Tree:
    """A parse node: a syntax rule applied to sub-trees, or a bare variable."""
    __slots__ = ("label", "typecode", "kids", "var")

    def __init__(self, label, typecode, kids=(), var=None):
        self.label = label          # syntax-rule label, or None for a variable
        self.typecode = typecode    # ’wff’ | ’class’ | ’setvar’
        self.kids = list(kids)
        self.var = var              # token, if this node IS a variable

    def tokens(self):
        """Serialise back to the flat token sequence."""
        if self.var is not None:
            return [self.var]
        out, k = [], 0
        for t in RULES[self.label][1]:
            if t in VARTYPE:
                out.extend(self.kids[k].tokens()); k += 1
            else:
                out.append(t)
        return out

    def size(self):
        return 1 + sum(k.size() for k in self.kids)

    def show(self, indent=0):
        pad = "  " * indent
        if self.var is not None:
            return "%s%s : %s\n" % (pad, self.var, self.typecode)
        s = "%s%s : %s\n" % (pad, self.label, self.typecode)
        for k in self.kids:
            s += k.show(indent + 1)
        return s

    def __repr__(self):
        return " ".join(self.tokens())


RULES = {}      # label -> (typecode, pattern tokens)
VARTYPE = {}    # variable token -> typecode
RULEIDX = {}    # (typecode, first constant | None) -> [(label, pat, minlen)]


def build_index(by_tc):
    """Bucket rules by the FIRST token of their pattern.

    Without this the parser tries every rule of a typecode at every span --
    set.mm has roughly a thousand ‘class‘ productions, so a forty-token
    statement costs about 1.6 million rule attempts and the corpus takes
    hours.  But a rule beginning with a constant can only match a span
    beginning with that same constant, and most set.mm productions begin with
    a constant such as ’(’ or a symbol.  Bucketing on it removes almost all of
    the attempts before they are made.

```

```
    Rules beginning with a VARIABLE have to be tried at every span and are
    kept in the None bucket.

    minlen is len(pattern): every element -- constant or variable -- consumes
    at least one token, so a span shorter than the pattern cannot match."""
    RULEIDX.clear()
    for tc, rules in by_tc.items():
        for lab, pat in rules:
            if not pat:
                continue
            first = None if pat[0] in VARTYPE else pat[0]
            RULEIDX.setdefault((tc, first), []).append((lab, pat, len(pat)))
    return RULEIDX


# ============================================================================
#  grammar extraction
# ============================================================================
def build_grammar(mm):
    """Every $a whose typecode is not '|-' is a production rule.

    A $f hypothesis types a variable.  Together these are a context-free
    grammar whose nonterminals are the typecodes."""
    RULES.clear(); VARTYPE.clear()
    for lab in mm.order:
        typ, data = mm.labels[lab]
        if typ == "$f":
            VARTYPE[data[1]] = data[0]
        elif typ == "$a":
            stat = data[3]
            if stat and stat[0] != "|-":
                RULES[lab] = (stat[0], list(stat[1:]))

    by_tc = defaultdict(list)
    for lab, (tc, pat) in RULES.items():
        by_tc[tc].append((lab, pat))
    build_index(by_tc)          # keep the first-token index in step
    return by_tc


# ============================================================================
#  parsing
# ============================================================================
def parse(tokens, typecode, by_tc, memo=None, all_parses=False, cap=4):
    """Parse tokens[0:] entirely as `typecode`.  Returns a Tree, or None.

    Bottom-up over spans with memoisation.  A rule matches a span when its
    constant tokens line up and each of its variables consumes a sub-span of
    that variable's own typecode.  Variables have no fixed width, so every
    split has to be tried; rules carry few variables so this stays cheap."""
    tokens = list(tokens)
    n = len(tokens)
    if memo is None:
        memo = {}

    if not RULEIDX:
        build_index(by_tc)
    vt = VARTYPE
    idx = RULEIDX

    _rf = {}

    def rules_for(tc, tok):
        """Rules that can start here: those beginning with this exact constant,
        plus those beginning with a variable.

        The concatenation is CACHED.  Building it per call was slower than the
        unbucketed scan it replaced -- set.mm's grammar is heavily infix
        (A e. B, A = B, A C_ B all begin with a variable), so the variable
        bucket is large and re-concatenating it at every span dominated."""
        key = (tc, tok)
        r = _rf.get(key)
        if r is None:
            a = idx.get(key) or ()
            b = idx.get((tc, None)) or ()
            r = (a + b) if (a and b) else (a or b or ())
            _rf[key] = r
        return r

    def go(i, j, tc):
        key = (i, j, tc)
        if key in memo:
            return memo[key]
        memo[key] = None                      # cycle guard
        found = []
        span = j - i

        # a single token that IS a variable of this typecode
        if span == 1 and vt.get(tokens[i]) == tc:
            found.append(Tree(None, tc, (), tokens[i]))

        if not (found and not all_parses):
```

```
            for lab, pat, minlen in rules_for(tc, tokens[i] if i < j else None):
                if span < minlen:              # every element eats >= 1 token
                    continue
                for kids in match(pat, 0, i, j):
                    found.append(Tree(lab, tc, kids))
                    if not all_parses:
                        break
                if found and not all_parses:
                    break
                if all_parses and len(found) >= cap:
                    break

        memo[key] = found if all_parses else (found[0] if found else None)
        return memo[key]

    def match(pat, p, i, j):
        """Yield lists of child trees for pat[p:] covering tokens[i:j]."""
        np_ = len(pat)
        if p == np_:
            if i == j:
                yield []
            return
        t = pat[p]
        if t not in vt:                        # a constant: must line up
            if i < j and tokens[i] == t:
                for rest in match(pat, p + 1, i + 1, j):
                    yield rest
            return
        tc = vt[t]
        # every remaining element after this one still needs >= 1 token
        tail = np_ - p - 1
        if p == np_ - 1:                       # last element: takes the rest
            sub = go(i, j, tc)
            if sub:
                for s in (sub if isinstance(sub, list) else [sub]):
                    yield [s]
            return
        for m in range(i + 1, j - tail + 1):
            sub = go(i, m, tc)
            if not sub:
                continue
            for s in (sub if isinstance(sub, list) else [sub]):
                for rest in match(pat, p + 1, m, j):
                    yield [s] + rest
                if not isinstance(sub, list):
                    break

    r = go(0, n, typecode)
    if all_parses:
        return r or []
    return r


def parse_statement(stat, by_tc):
    """Parse a full '|- ...' statement: drop the typecode, parse the rest."""
    if not stat:
        return None
    return parse(stat[1:], "wff", by_tc)


# ====================================================================
#  matching: which assertions could conclude a goal
# ====================================================================
def match_tree(pat, goal, subst):
    """One-way match: can `pat` (with variables) be instantiated to `goal`?

    Variables in the pattern bind to whole sub-trees of the goal.  A variable
    already bound must bind consistently.  This is matching, not unification:
    the goal is treated as ground."""
    if pat.var is not None:
        if pat.typecode != goal.typecode:
            return False
        prev = subst.get(pat.var)
        if prev is None:
            subst[pat.var] = goal
            return True
        return prev.tokens() == goal.tokens()
    if pat.label != goal.label or len(pat.kids) != len(goal.kids):
        return False
    return all(match_tree(a, b, subst) for a, b in zip(pat.kids, goal.kids))


def candidates(mm, by_tc, goal_tree, cache):
    """Assertions whose conclusion matches the goal.  These are the candidate
    last steps of a backward search, and their count is the branching factor
    that branching.py estimated by prefix indexing."""
    out = []
    for lab in mm.order:
        typ, data = mm.labels[lab]
        if typ not in ("$a", "$p"):
            continue
        stat = data[3]
```

```
        if not stat or stat[0] != "|-":
            continue
        t = cache.get(lab, ...)
        if t is ...:
            try:
                t = parse_statement(stat, by_tc)
            except (RecursionError, MMError):
                t = None
            cache[lab] = t
        if t is None:
            continue
        s = {}
        if match_tree(t, goal_tree, s):
            out.append((lab, s))
    return out


# ============================================================================
#  commands
# ============================================================================
SELFTEST = r"""
$c wff class |- ( ) -> e. _V $.
$v ph ps A B $.
wph $f wff ph $.
wps $f wff ps $.
cA  $f class A $.
cB  $f class B $.
cvv $a class _V $.
wcel $a wff A e. B $.
wi $a wff ( ph -> ps ) $.
ax1 $a |- ( ph -> ( ps -> ph ) ) $.
th $p |- ( A e. B -> ( _V e. B -> A e. B ) ) $= ? $.
"""


def cmd_selftest(_):
    print("\n" + "=" * 74)
    print("  setmm_grammar.py  --  selftest")
    print("=" * 74 + "\n")
    mm = MM(); mm.read(Toks(SELFTEST))
    by_tc = build_grammar(mm)
    print("  grammar: %d rules, %d typed variables" % (len(RULES), len(VARTYPE)))

    bad = 0
    cases = ["wcel", "wi", "ax1", "th"]
    for lab in cases:
        typ, data = mm.labels[lab]
        stat = data[3]
        tc = stat[0]
        t = parse_statement(stat, by_tc) if tc == "|-" else \
            parse(stat[1:], tc, by_tc)
        ok = t is not None and t.tokens() == list(stat[1:])
        bad += (not ok)
        print("  %-6s %-5s %-40s %s"
              % (lab, tc, " ".join(stat[1:])[:40], "ok" if ok else "<-- FAILED"))

    # matching: ax1's conclusion should match th's, binding ph and ps
    pat = parse_statement(mm.labels["ax1"][1][3], by_tc)
    goal = parse_statement(mm.labels["th"][1][3], by_tc)
    s = {}
    m = match_tree(pat, goal, s)
    print("\n  match ax1's conclusion against th's statement: %s"
          % ("ok" if m else "<-- FAILED"))
    if m:
        for v, t in sorted(s.items()):
            print("      %-4s := %s" % (v, " ".join(t.tokens())))
    bad += (not m)

    # a NON-match must be rejected
    s2 = {}
    bad_match = match_tree(goal, pat, s2)
    print("  reverse direction correctly rejected: %s"
          % ("ok" if not bad_match else "<-- FAILED, matcher is too permissive"))
    bad += bool(bad_match)

    print("\n  %s\n" % ("all checks passed" if not bad
                        else "%d CHECK(S) FAILED" % bad))
    return 0 if not bad else 1


def cmd_roundtrip(a):
    print("\n" + "=" * 74)
    print("  setmm_grammar.py  --  round-trip")
    print("=" * 74 + "\n")
    mm = load(a.file)
    by_tc = build_grammar(mm)
    print("\n  grammar: %s production rules, %s typed variables"
          % (f"{len(RULES):,}", f"{len(VARTYPE):,}"))

    labs = [l for l in mm.order
            if mm.labels[l][0] in ("$a", "$p")
            and mm.labels[l][1][3] and mm.labels[l][1][3][0] == "|-"]
```

```
    if a.sample:
        # Spread across the WHOLE corpus, not a prefix.  The first few thousand
        # set.mm statements are propositional and exercise a small corner of the
        # grammar; class abstractions, restricted quantifiers and the harder
        # constructors live later.  A prefix of 2,000 can pass while the grammar
        # is still wrong for most of the file.
        step = max(1, len(labs) // a.sample)
        labs = labs[::step][:a.sample]
        print("  sampling every %d-th statement across the corpus" % step)
    elif a.limit:
        labs = labs[:a.limit]
    print("  parsing %s '|-' statements...\n" % f"{len(labs):,}")

    t0 = time.time()
    ok = fail = err = 0
    shown = 0
    for i, lab in enumerate(labs, 1):
        stat = mm.labels[lab][1][3]
        try:
            t = parse_statement(stat, by_tc)
        except (RecursionError, MMError):
            err += 1; continue
        if t is None:
            fail += 1
            if shown < a.show:
                print("    NO PARSE  %-12s %s" % (lab, " ".join(stat[:16])))
                shown += 1
        elif t.tokens() != list(stat[1:]):
            fail += 1
            if shown < a.show:
                print("    MISMATCH  %-12s" % lab)
                print("       in  %s" % " ".join(stat[1:])[:80])
                print("       out %s" % " ".join(t.tokens())[:80])
                shown += 1
        else:
            ok += 1
        if a.progress and i % a.progress == 0:
            print("    %s/%s  (%.0f/s)"
                  % (f"{i:,}", f"{len(labs):,}", i / max(time.time() - t0, 1e-9)))

    dt = time.time() - t0
    print("\n  " + "-" * 70)
    print("  round-tripped %s" % f"{ok:,}")
    print("  FAILED        %s" % f"{fail:,}")
    print("  errored       %s" % f"{err:,}")
    print("  elapsed       %.1fs" % dt)
    print("  " + "-" * 70)
    if not fail and not err:
        print("""
  Every statement parsed and serialised back byte-identically.  The grammar
  extracted from the syntax axioms is therefore the grammar the corpus was
  written in, and the trees can be trusted for matching.
""")
    else:
        print("""
  Failures mean the extracted grammar is not the corpus's grammar.  Trees built
  with it would be wrong in ways a matcher cannot detect, so fix this before
  building anything on top.
""")
    return 0 if not (fail or err) else 1


def cmd_tree(a):
    mm = load(a.file)
    by_tc = build_grammar(mm)
    if a.label not in mm.labels:
        print("\n  %s not found\n" % a.label); return 1
    stat = mm.labels[a.label][1][3]
    print("\n  %s\n    %s\n" % (a.label, " ".join(stat)))
    t = parse_statement(stat, by_tc)
    if t is None:
        print("  NO PARSE\n"); return 1
    print(t.show(2))
    print("  tree size %d nodes, round-trip %s\n"
          % (t.size(), "ok" if t.tokens() == list(stat[1:]) else "FAILED"))
    return 0


def cmd_match(a):
    mm = load(a.file)
    by_tc = build_grammar(mm)
    if a.label not in mm.labels:
        print("\n  %s not found\n" % a.label); return 1
    stat = mm.labels[a.label][1][3]
    goal = parse_statement(stat, by_tc)
    if goal is None:
        print("\n  goal does not parse\n"); return 1

    print("\n" + "=" * 74)
    print("  BACKWARD CANDIDATES for %s" % a.label)
    print("=" * 74)
    print("\n  goal: %s\n" % " ".join(stat))
```

```
    print("  scanning %s assertions..." % f"{len(mm.order):,}")
    t0 = time.time()
    cands = candidates(mm, by_tc, goal, {})
    print("  %.1fs\n" % (time.time() - t0))
    print("  %d assertions could conclude this goal\n" % len(cands))
    for lab, s in cands[:a.limit or 25]:
        print("    %-14s %s" % (lab, " ".join(mm.labels[lab][1][3])[:64]))
    print("""
  That count IS the backward branching factor at this node -- measured by
  matching rather than estimated by prefix indexing.  It is the number a
  goal-directed prover faces, and the number a learned ranker would reorder.
""")
    return 0


def main():
    ap = argparse.ArgumentParser(prog="setmm_grammar", description=__doc__,
            formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd")
    sub.add_parser("selftest")
    r = sub.add_parser("roundtrip"); r.add_argument("file", nargs="?", default="set.mm")
    r.add_argument("--limit", type=int, default=0)
    r.add_argument("--sample", type=int, default=0,
                   help="N statements spread across the whole corpus "
                        "(better coverage than a prefix of N)")
    r.add_argument("--show", type=int, default=10)
    r.add_argument("--progress", type=int, default=5000)
    t = sub.add_parser("tree"); t.add_argument("file", nargs="?", default="set.mm")
    t.add_argument("label")
    m = sub.add_parser("match"); m.add_argument("file", nargs="?", default="set.mm")
    m.add_argument("label"); m.add_argument("--limit", type=int, default=25)

    a = ap.parse_args()
    if a.cmd == "selftest":    sys.exit(cmd_selftest(a))
    elif a.cmd == "roundtrip": sys.exit(cmd_roundtrip(a))
    elif a.cmd == "tree":      sys.exit(cmd_tree(a))
    elif a.cmd == "match":     sys.exit(cmd_match(a))
    else:                      ap.print_help()


def _run_with_big_stack(fn):
    """Run fn on a thread with a large stack.

    sys.setrecursionlimit only raises PYTHON's guard; the C stack is a separate
    and much harder limit.  Raising the first without the second converts a
    clean RecursionError into a process-killing stack overflow -- no traceback,
    the shell simply exits.  That is what a deeply nested set.mm statement did
    to this parser.  A 256 MB thread stack makes the raised limit honest."""
    import threading
    try:
        threading.stack_size(256 * 1024 * 1024)
    except (ValueError, RuntimeError):
        try:
            threading.stack_size(64 * 1024 * 1024)
        except Exception:
            pass
    box = {}

    def target():
        try:
            box["rc"] = fn()
        except SystemExit as e:
            box["rc"] = e.code
        except RecursionError:
            print("\n  RecursionError: a statement nested deeper than the "
                  "stack allows.\n  This is the CLEAN failure -- the process "
                  "survived.\n")
            box["rc"] = 2
    t = threading.Thread(target=target, daemon=True)
    t.start()
    # join in slices so Ctrl-C reaches the main thread.  A bare join() blocks
    # uninterruptibly and a non-daemon worker keeps the process alive after it,
    # which is why Ctrl-C did nothing during a long search.
    try:
        while t.is_alive():
            t.join(0.2)
    except KeyboardInterrupt:
        print("\n  interrupted.\n")
        return 130
    return box.get("rc", 0)


if __name__ == "__main__":
    sys.exit(_run_with_big_stack(main) or 0)
```

C.3 `predator71.py` — the prover

```
#!/usr/bin/env python3
r"""
Predator_7.1 -- a prover for set.mm.  Unification, metavariables, checked proofs.
```

```
Brian Tenneson.

    python predator71.py selftest                    no files needed
    python predator71.py prove set.mm --label prcom  attempt a proof
    python predator71.py verify prcom_p71.mm         check it independently

WHAT CHANGED FROM 7.0, AND WHY
------------------------------
7.0 searched by MATCHING: the goal was ground, the assertion's conclusion held
the variables, and the match was one-way.  That works only for assertions whose
hypothesis variables all occur in their conclusion.

ax-mp fails that test.  Its conclusion is the bare variable `ps`, so matching it
against a goal binds ps and leaves `ph` -- the entire antecedent -- with no
value.  syl fails the same way.  Those are the two most-used logical labels in
set.mm, and 864 of 4,643 assertions before prcom were excluded with them.

7.1 replaces matching with UNIFICATION over two variable namespaces:

    assertion variables   ph, ps, A, B ...   renamed apart at each use
    metavariables         ?1, ?2, ...        stand for not-yet-known subterms

Applying ax-mp to goal G binds ps := G and leaves ph unknown, so ph becomes a
metavariable ?1 and the second subgoal is  ( ?1 -> G ).  That subgoal is NOT
unconstrained: its consequent is fixed, so only assertions concluding something
of the form ( _ -> G ) can match, and whichever one does determines ?1.

That is the whole idea.  A free metavariable is harmless as long as a later step
determines it; what matters is that no step is required to INVENT a formula out
of nothing.

PROOFS ARE EMITTED AND CHECKED
------------------------------
A parse tree is a Metamath syntax proof: a node with rule L and children k1..kn
serialises in RPN as  proof(k1) ... proof(kn) L.  So the 95% of set.mm proof
steps that are formula construction are GENERATED from the tree rather than
searched for.

A logical step applying assertion A under substitution sigma emits

    [ trees for A's mandatory $f hypotheses, under sigma ]
    [ recursive proofs of A's $e hypotheses ]
    A

which is exactly what the verifier consumes.  `prove` writes a .mm file;
nothing here reports its own success.

WHAT IS STILL NOT DONE
----------------------
Disjoint-variable conditions are checked by the verifier but are not used to
PRUNE the search, so 7.1 can spend effort on branches the verifier will later
reject.  Any proof it emits is still sound -- the verifier is the authority --
but the search is not as sharp as it could be.
"""
from __future__ import annotations
import argparse, heapq, itertools, os, sys, time
from collections import defaultdict

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
try:
    from metamath import MM, Toks, load, MMError
    import setmm_grammar as G
except ImportError as e:
    raise SystemExit("predator71.py needs metamath.py and setmm_grammar.py "
                     "in the same folder (%s)" % e)

VERSION = "7.1"
WORKDIR = r"C:\google drive\Automated Theorem Proving"
if os.path.isdir(WORKDIR):
    os.chdir(WORKDIR)
sys.setrecursionlimit(8000)


# ============================================================================
#  trees with two flexible namespaces
# ============================================================================
_counter = itertools.count(1)


def fresh(tc):
    """A new metavariable of the given typecode."""
    return G.Tree(None, tc, (), "?%d" % next(_counter))


def is_meta(t):
    return t.var is not None and t.var.startswith("?")


def rename_apart(t, mapping):
    """Fresh metavariables for an assertion's own variables.

    Without this, two uses of the same assertion in one proof would share
```

```
    variable names and unify with each other by accident."""
    if t.var is not None:
        if t.var not in mapping:
            mapping[t.var] = fresh(t.typecode)
        return mapping[t.var]
    return G.Tree(t.label, t.typecode, [rename_apart(k, mapping) for k in t.kids])


def walk(t, sub):
    while is_meta(t) and t.var in sub:
        t = sub[t.var]
    return t


def apply_sub(t, sub):
    t = walk(t, sub)
    if t.var is not None:
        return t
    return G.Tree(t.label, t.typecode, [apply_sub(k, sub) for k in t.kids])


def occurs(v, t, sub):
    t = walk(t, sub)
    if t.var is not None:
        return t.var == v
    return any(occurs(v, k, sub) for k in t.kids)


def unify(a, b, sub):
    """Robinson unification over metavariables, with occurs check.

    Only metavariables are flexible.  A statement's ordinary variables have
    already been renamed to metavariables by rename_apart, so everything
    flexible is a metavariable and everything else is rigid."""
    a, b = walk(a, sub), walk(b, sub)
    if a is b:
        return sub
    if is_meta(a):
        if is_meta(b) and a.var == b.var:
            return sub
        if a.typecode != b.typecode:
            return None
        if occurs(a.var, b, sub):
            return None
        s = dict(sub); s[a.var] = b; return s
    if is_meta(b):
        return unify(b, a, sub)
    if a.var is not None or b.var is not None:
        return sub if (a.var == b.var and a.typecode == b.typecode) else None
    if a.label != b.label or len(a.kids) != len(b.kids):
        return None
    for x, y in zip(a.kids, b.kids):
        sub = unify(x, y, sub)
        if sub is None:
            return None
    return sub


def ground(t, sub, fallback):
    """Instantiate any surviving metavariable.

    A metavariable never determined by the search is genuinely unconstrained --
    any well-formed term of its typecode makes the proof valid -- so it is
    filled with a declared variable.  set.mm statements are schematic, so this
    keeps the emitted proof well formed."""
    t = walk(t, sub)
    if t.var is not None:
        return fallback[t.typecode] if is_meta(t) else t
    return G.Tree(t.label, t.typecode, [ground(k, sub, fallback) for k in t.kids])


# ============================================================================
#  emitting Metamath proofs
# ============================================================================
def tree_proof(t, fvar):
    """RPN for a parse tree: children first, then the rule label.

    This is the formula-construction majority of a set.mm proof, produced by
    walking a tree instead of searching for it."""
    if t.var is not None:
        if t.var not in fvar:
            raise MMError("no $f hypothesis for variable %s" % t.var)
        return [fvar[t.var]]
    out = []
    for k in t.kids:
        out.extend(tree_proof(k, fvar))
    out.append(t.label)
    return out


class Step:
    __slots__ = ("label", "fmap", "subs", "data")
```

```
188
189     def __init__(self, label, fmap, data):
190         self.label, self.fmap, self.data = label, fmap, data
191         # One slot per $e hypothesis, filled BY INDEX.  Appending in the order
192         # subgoals happen to be solved is wrong once the search picks the most
193         # constrained goal first rather than the leftmost: emit() must lay the
194         # hypotheses out in declaration order or the verifier reports a
195         # hypothesis mismatch.
196         self.subs = [None] * len(data[2])
197
198     def emit(self, sub, fvar, fallback):
199         _, f_hyps, e_hyps, _ = self.data
200         out = []
201         for _, tc, var in f_hyps:
202             t = self.fmap.get(var)
203             if t is None:
204                 raise MMError("%s: unbound %s" % (self.label, var))
205             out.extend(tree_proof(ground(t, sub, fallback), fvar))
206         for i, s in enumerate(self.subs):
207             if s is None:
208                 raise MMError("%s: hypothesis %d never solved" % (self.label, i))
209             out.extend(s.emit(sub, fvar, fallback))
210         out.append(self.label)
211         return out
212
213
214 # ====================================================================
215 #  the index
216 # ====================================================================
217 class Index:
218     """Assertions with their conclusion parsed, bucketed by conclusion head.
219
220     Unlike 7.0 this excludes nothing: an assertion whose hypotheses mention
221     variables absent from its conclusion is exactly the case metavariables
222     exist to handle."""
223
224     def __init__(self, mm, by_tc, upto=None, say=None):
225         self.by_tc = by_tc
226         self.closers = defaultdict(list)   # no $e hypotheses: end a branch
227         self.openers = defaultdict(list)   # have $e hypotheses: extend it
228         self.n = 0
229         order = mm.order[:upto] if upto is not None else mm.order
230         for lab in order:
231             typ, data = mm.labels[lab]
232             if typ not in ("$a", "$p"):
233                 continue
234             concl = data[3]
235             if not concl or concl[0] != "|-" or len(concl) < 2:
236                 continue
237             try:
238                 t = G.parse(concl[1:], "wff", by_tc)
239             except (RecursionError, MMError):
240                 t = None
241             if t is None:
242                 continue
243             self.n += 1
244             head = None if t.var is not None else t.label
245             # Split on whether the assertion has $e hypotheses.  One with none
246             # that unifies CLOSES the goal outright -- it is a complete proof
247             # in one step.  Those must never be missed to a candidate cap, and
248             # there are few of them, so they are kept separate and always tried
249             # in full.  axin1, falanfal and tbw-ax4 each need exactly one such
250             # assertion (pm2.01, anidm, falim) and all three failed at 20,000
251             # expansions because the cap took an arbitrary 48 of thousands.
252             (self.closers if not data[2] else self.openers)[head].append(
253                 (lab, t, data))
254         if say:
255             nc = sum(len(v) for v in self.closers.values())
256             say("    %s assertions indexed (%s close a goal outright)"
257                 % (f"{self.n:,}", f"{nc:,}"))
258
259     def candidates(self, goal):
260         """(closers, openers) that could unify with this goal.
261
262         A conclusion headed by a rule can only meet a goal headed by the same
263         rule.  A conclusion that is a bare variable -- ax-mp -- meets anything,
264         and so does any goal that is itself a metavariable.
265
266         Closers are returned separately because they are tried exhaustively:
267         a closer that unifies finishes the branch, so skipping one loses a
268         proof outright, whereas skipping an opener only loses a route."""
269         def grab(d):
270             if goal.var is not None:
271                 return [x for b in d.values() for x in b]
272             return d.get(goal.label, []) + d.get(None, [])
273         return grab(self.closers), grab(self.openers)
274
275
276 # ====================================================================
277 #  the search
278 # ====================================================================
279 class Node:
```

```
    __slots__ = ("goals", "sub", "trail", "depth")

    def __init__(self, goals, sub, trail, depth):
        self.goals, self.sub, self.trail, self.depth = goals, sub, trail, depth


def n_metas(t, sub, acc=None):
    """How many distinct metavariables survive in this goal."""
    if acc is None:
        acc = set()
    t = walk(t, sub)
    if t.var is not None:
        if is_meta(t):
            acc.add(t.var)
        return acc
    for k in t.kids:
        n_metas(k, sub, acc)
    return acc


def pick_goal(goals, sub):
    """Choose the MOST CONSTRAINED open goal, not simply the first.

    A goal that is a bare metavariable constrains nothing -- every assertion in
    the corpus unifies with it -- so expanding it is pure guessing.  Left to
    itself the search does exactly that: ax-mp turns goal G into ( ?1 -> G )
    with ?1 free, and ?1 can be attacked by ax-mp again, giving an infinite
    regress that breadth-first order explores forever.

    Deferring under-constrained goals lets a SIBLING goal bind their
    metavariables first.  This is the standard fix for backward modus ponens
    and it is what makes the difference between thrashing and terminating."""
    best, bi = None, 0
    for i, (g, slot, _hix) in enumerate(goals):
        gg = walk(g, sub)
        bare = 1 if (gg.var is not None and is_meta(gg)) else 0
        key = (bare, len(n_metas(gg, sub)), -gg.size())
        if best is None or key < best:
            best, bi = key, i
    return bi


def prove(goal_tree, index, budget, max_depth, rank=None, say=print,
          progress=2000, max_open=6):
    """Backward search with metavariables.

    A state is (open goals, substitution).  Expanding the first goal picks an
    assertion, unifies its renamed conclusion with the goal, and replaces the
    goal by that assertion's hypotheses under the resulting substitution.  Any
    variable of the assertion not determined by the unification survives as a
    metavariable in those subgoals, to be determined by a later step.

    The substitution is threaded through every open goal, so binding a
    metavariable in one branch constrains the others.  That is what stops the
    search inventing formulas."""
    start = Node([(goal_tree, None, 0)], {}, (), 0)
    frontier = [(0.0, 0, start)]
    exp = tie = 0
    seen = set()
    t0 = time.perf_counter()
    while frontier:
        _, _, node = heapq.heappop(frontier)
        exp += 1
        if exp > budget:
            return None, exp
        if progress and say and exp % progress == 0:
            say("      %s expansions, %d open goals, %.0fs"
                % (f"{exp:,}", len(node.goals), time.perf_counter() - t0))
        if not node.goals:
            root = None
            for parent, ix, st in node.trail:
                if parent is None:
                    root = st
                else:
                    parent.subs[ix] = st
            return (root, node.sub), exp
        if node.depth >= max_depth:
            continue

        if len(node.goals) > max_open:
            continue                        # too many speculative subgoals open
        gi = pick_goal(node.goals, node.sub)
        (gt, slot, hix) = node.goals[gi]
        rest = node.goals[:gi] + node.goals[gi + 1:]
        gt = apply_sub(gt, node.sub)

        key = (node.depth, " ".join(gt.tokens()),
               tuple(sorted(" ".join(apply_sub(g, node.sub).tokens())
                            for g, _, _ in rest)))
        if key in seen:
            continue
        seen.add(key)
```

```

        closers, openers = index.candidates(gt)
        # Every closer, then a capped slice of the openers.  Ordering by
        # subgoal count is not a heuristic preference -- a closer that unifies
        # IS a proof of this goal, so trying openers first would be searching
        # past the answer.
        sc_c = rank(gt, closers) if rank else [0.0] * len(closers)
        sc_o = rank(gt, openers) if rank else [0.0] * len(openers)
        pick = ([closers[i] for i in
                 sorted(range(len(closers)), key=lambda i: -sc_c[i])] +
                [openers[i] for i in
                 sorted(range(len(openers)), key=lambda i: -sc_o[i])][:48])
        scored_of = dict()

        for lab, ct, data in pick:
            m = {}
            c2 = rename_apart(ct, m)
            s2 = unify(c2, gt, node.sub)
            if s2 is None:
                continue
            _, f_hyps, e_hyps, _ = data
            fmap = {var: m.get(var, fresh(tc)) for _, tc, var in f_hyps}
            for _, tc, var in f_hyps:
                m.setdefault(var, fmap[var])
            step = Step(lab, fmap, data)
            newgoals = []
            ok = True
            for hj, (_, stat) in enumerate(e_hyps):
                try:
                    ht = G.parse(stat[1:], "wff", index.by_tc)
                except (RecursionError, MMError):
                    ht = None
                if ht is None:
                    ok = False; break
                newgoals.append((rename_apart(ht, m), step, hj))
            if not ok:
                continue
            tie += 1
            heapq.heappush(frontier,
                           (node.depth + 1 - 0.0, tie,
                            Node(newgoals + rest, s2,
                                 node.trail + ((slot, hix, step),),
                                 node.depth + 1)))
    return None, exp


# ======================================================================
#  selftest
# ======================================================================
SELFTEST = r"""
$c wff |- ( ) -> /\ $.
$v ph ps ch $.
wph $f wff ph $.
wps $f wff ps $.
wch $f wff ch $.
wi $a wff ( ph -> ps ) $.
wa $a wff ( ph /\ ps ) $.
ax1 $a |- ( ph -> ( ps -> ph ) ) $.
ax2 $a |- ( ( ph -> ( ps -> ch ) ) -> ( ( ph -> ps ) -> ( ph -> ch ) ) ) $.
${ mpmin $e |- ph $.
   mpmaj $e |- ( ph -> ps ) $.
   ax-mp $a |- ps $. $}
"""


def cmd_selftest(a):
    print("\n" + "=" * 74)
    print("  PREDATOR_7.1 v%s  --  selftest" % VERSION)
    print("=" * 74 + "\n")
    mm = MM(); mm.read(Toks(SELFTEST))
    by_tc = G.build_grammar(mm)
    fvar, fallback = {}, {}
    for lab in mm.order:
        typ, d = mm.labels[lab]
        if typ == "$f":
            fvar[d[1]] = lab
            fallback.setdefault(d[0], G.Tree(None, d[0], (), d[1]))

    idx = Index(mm, by_tc, say=lambda s: print("  " + s))
    print("  ax-mp is included: its conclusion is the bare variable `ps`,")
    print("  which 7.0 excluded as undetermined.\n")

    bad = 0

    # (1) the identity  |- ( ph -> ph ).  Its only route is two applications of
    #     ax-mp, which 7.0 could not use at all.
    goal_toks = "( ph -> ph )".split()
    gt = G.parse(goal_toks, "wff", by_tc)
    print("  [1] goal  |- %s" % " ".join(goal_toks))
    print("      route requires ax-mp twice; 7.0 could not attempt this")
    t0 = time.perf_counter()
    res, exp = prove(gt, idx, 20000, 8)
```

```
    dt = time.perf_counter() - t0
    if res is None:
        print("      NOT PROVED, %s expansions  <-- FAILED" % f"{exp:,}"); bad += 1
    else:
        root, sub = res
        proof = root.emit(sub, fvar, fallback)
        print("      proved: %s expansions, %.2fs, %d proof steps"
              % (f"{exp:,}", dt, len(proof)))
        src = SELFTEST + "\nchk $p |- %s $= %s $.\n" % (
            " ".join(goal_toks), " ".join(proof))
        m2 = MM()
        try:
            m2.read(Toks(src))
            r = m2.verify("chk")
            print("      metamath.py verify: %s" % r.upper())
            bad += (r != "ok")
        except MMError as e:
            print("      metamath.py verify: FAILED -- %s" % e); bad += 1

    print("\n  %s\n" % ("all checks passed" if not bad
                        else "%d CHECK(S) FAILED" % bad))
    return 0 if not bad else 1


# ======================================================================
#  prove
# ======================================================================
def cmd_easiest(a):
    """Rank set.mm theorems by LOGICAL proof length, shortest first.

    Raw proof length is 95% formula construction, so sorting by it ranks
    theorems by how verbose their notation is.  Counting only |- steps ranks
    them by how much reasoning they need, which is what a prover faces.

    A theorem needing one logical step is the easiest thing this program can
    be asked to do.  If it fails there, the problem is not the budget."""
    from metamath import classify
    print("\n" + "=" * 74)
    print("  EASIEST TARGETS  --  by logical proof length")
    print("=" * 74 + "\n")
    mm = load(a.file)
    kind = classify(mm)

    thms = [l for l in mm.order if mm.labels[l][0] == "$p"]
    if a.scan:
        thms = thms[:a.scan]
    print("\n  measuring %s proofs..." % f"{len(thms):,}")
    rows = []
    for i, lab in enumerate(thms, 1):
        try:
            proof = mm.decompress(lab, mm.proofs[lab])
        except (MMError, RecursionError, ValueError):
            continue
        if "?" in proof:
            continue
        n = sum(1 for st in proof if kind.get(st) == "logic")
        if 0 < n <= a.max_steps:
            rows.append((n, len(proof), lab))
        if a.progress and i % a.progress == 0:
            print("    %s/%s" % (f"{i:,}", f"{len(thms):,}"))

    rows.sort()
    print("\n  %s theorems need %d or fewer logical steps\n"
          % (f"{len(rows):,}", a.max_steps))
    print("  %-5s %-7s %-14s %s" % ("logic", "raw", "label", "statement"))
    print("  " + "-" * 68)
    for n, raw, lab in rows[:a.limit]:
        stat = " ".join(mm.labels[lab][1][3])
        print("  %-5d %-7d %-14s %s" % (n, raw, lab, stat[:40]))
    print("""
  The `logic` column is the number of reasoning steps; `raw` includes the
  formula construction.  Their ratio is the notation factor for that theorem.

  Start at the top.  A one-logical-step theorem is one assertion applied once,
  and if the search cannot find that, no budget will help.
""")
    return 0


def cmd_prove(a):
    print("\n" + "=" * 74)
    print("  PREDATOR_7.1 v%s  --  prove %s" % (VERSION, a.label))
    print("=" * 74 + "\n")
    mm = load(a.file)
    by_tc = G.build_grammar(mm)
    if a.label not in mm.labels:
        print("\n  %s not found\n" % a.label); return 1
    stat = mm.labels[a.label][1][3]
    print("\n  goal  %s" % " ".join(stat))

    fvar, fallback = {}, {}
    for lab in mm.order:
```

```
        typ, d = mm.labels[lab]
        if typ == "$f":
            fvar.setdefault(d[1], lab)
            fallback.setdefault(d[0], G.Tree(None, d[0], (), d[1]))

    cut = mm.order.index(a.label)
    print("\n  indexing assertions before %s (parses conclusions once)..." % a.label)
    t0 = time.perf_counter()
    idx = Index(mm, by_tc, upto=cut, say=lambda s: print("  " + s))
    print("    %.1fs" % (time.perf_counter() - t0))

    gt = G.parse(stat[1:], "wff", by_tc)
    if gt is None:
        print("\n  goal does not parse\n"); return 1

    print("\n  searching (budget %s, max depth %d)..."
          % (f"{a.budget:,}", a.max_depth))
    t0 = time.perf_counter()
    res, exp = prove(gt, idx, a.budget, a.max_depth)
    dt = time.perf_counter() - t0

    if res is None:
        print("\n  NOT PROVED.  %s expansions, %.1fs" % (f"{exp:,}", dt))
        print("""
  Separate the causes before tuning:
    * budget exhausted        -> raise --budget
    * proof deeper than limit -> raise --max-depth
    * no route through the indexed assertions at all
""")
        return 1

    root, sub = res
    try:
        proof = root.emit(sub, fvar, fallback)
    except MMError as e:
        print("\n  found a derivation but could not emit it: %s\n" % e)
        return 1

    print("\n  PROVED.  %s expansions, %.1fs, %s proof steps"
          % (f"{exp:,}", dt, f"{len(proof):,}"))
    out = a.out or ("%s_p71.mm" % a.label)
    with open(out, "w") as f:
        f.write("$( Predator_7.1 proof of %s $)\n" % a.label)
        f.write("$[ %s $]\n" % a.file)
        f.write("chk $p %s $= %s $.\n" % (" ".join(stat), " ".join(proof)))
    print("  wrote %s" % out)

    # Verify immediately, in process.  This is not self-reporting: it is the
    # CV's own stack machine, which knows nothing about how the proof was
    # found.  Writing the file and telling the user to check it is still the
    # primary claim; this just fails fast when the proof is wrong.
    print("\n  handing the certificate to the CV...")
    try:
        chk = MM()
        chk.labels = dict(mm.labels)
        chk.order = list(mm.order)
        chk.proofs = dict(mm.proofs)
        chk.constants, chk.variables = mm.constants, mm.variables
        chk.scope_dvs = dict(mm.scope_dvs)
        dvs, f_hyps, e_hyps, _ = mm.labels[a.label][1]
        chk.labels["__chk__"] = ("$p", (dvs, f_hyps, e_hyps, stat))
        chk.proofs["__chk__"] = proof
        chk.scope_dvs["__chk__"] = mm.scope_dvs.get(a.label, dvs)
        r = chk.verify("__chk__")
        print("  CV verdict: %s" % r.upper())
        if r != "ok":
            return 1
    except MMError as e:
        print("  CV verdict: FAILED -- %s" % e)
        return 1
    print("""
  This program's word is not evidence.  Check it:

      python metamath.py verify %s
""" % out)
    return 0


def main():
    ap = argparse.ArgumentParser(prog="predator71", description=__doc__,
            formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd")
    sub.add_parser("selftest")
    e = sub.add_parser("easiest")
    e.add_argument("file", nargs="?", default="set.mm")
    e.add_argument("--max-steps", type=int, default=2,
                   help="keep theorems needing at most this many logical steps")
    e.add_argument("--scan", type=int, default=8000,
                   help="how many theorems to measure (0 = all)")
    e.add_argument("--limit", type=int, default=30)
    e.add_argument("--progress", type=int, default=2000)
    p = sub.add_parser("prove")
```

```
    p.add_argument("file", nargs="?", default="set.mm")
    p.add_argument("--label", required=True)
    p.add_argument("--budget", type=int, default=50000)
    p.add_argument("--max-depth", type=int, default=10)
    p.add_argument("--out", default=None)

    a = ap.parse_args()
    if a.cmd == "selftest":  sys.exit(cmd_selftest(a))
    elif a.cmd == "easiest": sys.exit(cmd_easiest(a))
    elif a.cmd == "prove":   sys.exit(cmd_prove(a))
    else:                    ap.print_help()


def _big_stack(fn):
    import threading
    try:
        threading.stack_size(256 * 1024 * 1024)
    except (ValueError, RuntimeError):
        pass
    box = {}

    def target():
        try:
            box["rc"] = fn()
        except SystemExit as e:
            box["rc"] = e.code
        except RecursionError:
            print("\n  RecursionError -- a term nested deeper than the stack.\n")
            box["rc"] = 2
    t = threading.Thread(target=target, daemon=True)
    t.start()
    try:
        while t.is_alive():
            t.join(0.2)
    except KeyboardInterrupt:
        print("\n  interrupted.\n")
        return 130
    return box.get("rc", 0)


if __name__ == "__main__":
    sys.exit(_big_stack(main) or 0)
```

C.4 `tau.py` — transaction and materialisation counts

```
#!/usr/bin/env python3
r"""
tau.py -- transaction counts, derived rules, and checked proofs.
Brian Tenneson.  Companion to predator5.py.

    python tau.py sic                the canonical rule-enumeration SIC
    python tau.py compile            thm:compilation, measured
    python tau.py prove --target ...  find a proof AND check it
    python tau.py axes               the two speedup axes, side by side

WHY THIS FILE EXISTS
--------------------
*Depths of a Simulation* v45 measures speedup by the TRANSACTION COUNT

    tau_M(Gamma, phi) = min { m : phi in C(Gamma, m) }

which counts STAGE ADVANCES.  Predator_5 does not reduce it and cannot:
reordering Sigma(p,m) leaves the proof at the same depth, and the corollary to
thm:branchcovering says no target-search SIC reaches phi before ||phi||.  What
Predator_5 reduces is the work performed INSIDE a stage -- how many states are
materialised before the target appears.

So there are two axes, and v45 formalises one:

    axis                 quantity          lowered by          in v45
    ---------------------------------------------------------------------
    stages               tau               derived rules       thm:compilation
    work per stage       materialisations  reordering Sigma    --

This file implements the first.  thm:compilation says any finite family of
provable targets can be compiled into derived rules that collapse their
transaction counts to tau <= 2.  That is a theorem about the COMPILED targets.
The interesting question, which the theorem does not answer, is what happens to
targets the cache was NOT built for.  `compile` measures it.

THE OBJECT BEING RUN IS NOT THE SEARCH TREE
-------------------------------------------
predator5.sweep() builds a graph whose nodes are SETS of formulas -- the
novelty state graph, which is the right object for a search.

The canonical rule-enumeration SIC E_{F,Y} of def:canonicalenum is different
and much simpler.  It has exactly ONE state per stage, and that state only
grows:

    C(Gamma, 1)   = Gamma
```

```
    C(Gamma, m+1) = D_F(C(Gamma, m))

D_F fires every rule on every available tuple at once.  This is v45's formal
description of brute force, and tau is its stage index.  Conflating the two
objects is easy and would make every number here wrong, so they are kept
separate: sic_stages() below is E_{F,Y}, and predator5.sweep() is the search.

PROOFS ARE EMITTED AND CHECKED
------------------------------
predator5 reports a length and an expansion count.  This file extracts the
actual proof and re-checks it against the rule, independently of the search
that found it: every step must be a genuine condensed detachment whose premises
are already present, and the last line must be the target.  A search that
reports success and a proof that checks are different claims.
"""
from __future__ import annotations
import argparse, heapq, json, os, sys, time
from collections import defaultdict, deque

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
try:
    import predator5 as P5
    from predator5 import (GAMMA, NAMED, show, detach, admissible, normalise,
                           split_targets, size, is_var)
except ImportError as e:
    raise SystemExit("tau.py needs predator5.py in the same folder (%s)" % e)

WORKDIR = r"C:\google drive\Automated Theorem Proving"
if os.path.isdir(WORKDIR):
    os.chdir(WORKDIR)


# ============================================================================
#  E_{F,Y} -- the canonical rule-enumeration SIC  (def:canonicalenum)
# ============================================================================
def sic_stages(gamma, max_stage, max_size, rules=None, cap=None, say=None):
    """C(Gamma,1) = Gamma;  C(Gamma,m+1) = D_F(C(Gamma,m)).

    ONE state per stage, growing.  Returns [C_1, C_2, ...] as frozensets.

    `rules` is a set of derived rules in the sense of v45: each is a partial
    map from a tuple of formulas to a formula, sound because the tuple proves
    the value.  Here a derived rule is keyed on Gamma itself -- d(Gamma) = phi
    for a compiled target phi -- which is the construction in the proof of
    thm:compilation.

    `cap` bounds the pairs tried per stage, so the sweep terminates on a
    fragment whose closure is large.  It is a truncation of D_F and is
    reported, because a truncated D_F is no longer the canonical machine."""
    cur = frozenset(gamma)
    out = [cur]
    truncated = False
    for m in range(1, max_stage):
        items = sorted(cur, key=lambda t: (size(t), t))
        new = set()
        tried = 0
        for major in items:
            if is_var(major):
                continue
            for minor in items:
                tried += 1
                if cap and tried > cap:
                    truncated = True
                    break
                c = detach(major, minor, max_size)
                if c is not None and c not in cur:
                    new.add(c)
            if cap and tried > cap:
                break
        if rules:
            # a derived rule fires as soon as its premise tuple is present.
            # Compiled targets are keyed on Gamma, so they all fire at stage 2.
            for prem, val in rules.items():
                if val not in cur and all(p in cur for p in prem):
                    new.add(val)
        if not new:
            break
        cur = cur | new
        out.append(cur)
        if say:
            say("    stage %d: %s formulas (+%d)%s"
                % (m + 1, f"{len(cur):,}", len(new),
                   "  [D_F truncated]" if truncated else ""))
    return out


def tau(stages, target):
    """tau_M(Gamma, phi) = min { m : phi in C(Gamma,m) }, 1-indexed as in v45."""
    for i, s in enumerate(stages, start=1):
        if target in s:
            return i
    return None
```

```
138
139
140 # ============================================================================
141 #  derived rules  (thm:compilation)
142 # ============================================================================
143 def compile_rules(gamma, targets):
144     """Build D as in the proof of thm:compilation.
145
146     For each provable target phi, the proof uses finitely many premises from
147     Gamma; take the tuple of those actually available and define d(tuple) = phi.
148     Since the tuple proves phi, d is a derived rule and F^D is conservative
149     (the proposition preceding thm:compilation).
150
151     Keyed on the whole of Gamma here, which is the coarsest sound choice and
152     makes every compiled target available at stage 2 -- exactly the bound the
153     theorem asserts."""
154     key = tuple(sorted(gamma, key=lambda t: (size(t), t)))
155     return {key: t for t in targets}
156
157
158 # ============================================================================
159 #  proofs: extract, then CHECK independently
160 # ============================================================================
161 def search_with_path(gamma, target, policy, max_size, edge_cap, budget,
162                      lam=1.0):
163     """predator5.guided_search, but returning the path that reached the target.
164
165     Returns (length, expansions, path) where path is a list of Edge objects in
166     the order applied.  predator5 discards this; a proof cannot be checked
167     without it."""
168     start = frozenset(gamma)
169     if target in start:
170         return 0, 0, []
171     seen = {start}
172     parent = {}
173     frontier = [(0.0, 0, 0, start)]
174     exp = tie = 0
175     while frontier:
176         _, depth, _, p = heapq.heappop(frontier)
177         exp += 1
178         if exp > budget:
179             return None, exp, []
180         edges, sc = policy.order(p, admissible(p, max_size, edge_cap),
181                                  gamma, target)
182         for e, s in zip(edges, sc):
183             qs = p | {e.concl}
184             if qs in seen:
185                 continue
186             seen.add(qs)
187             parent[qs] = (p, e)
188             if e.concl == target:
189                 path, cur = [], qs
190                 while cur in parent:
191                     pv, ed = parent[cur]
192                     path.append(ed)
193                     cur = pv
194                 return depth + 1, exp, list(reversed(path))
195             tie += 1
196             heapq.heappush(frontier, (depth + 1 - lam * s, depth + 1, tie, qs))
197     return None, exp, []
198
199
200 def check_proof(gamma, path, target, max_size):
201     """Replay the proof against the rule, independently of the search.
202
203     Each step must be a genuine condensed detachment whose two premises are
204     already present, its stated conclusion must be what the rule actually
205     produces, and the final line must be the target.  Returns (ok, message).
206
207     This is the difference between a search reporting success and a proof
208     that checks.  It re-derives every step from `detach` rather than trusting
209     the Edge object the search built."""
210     state = set(gamma)
211     for i, e in enumerate(path, 1):
212         if e.major not in state:
213             return False, "step %d: major premise %s not available" % (i, show(e.major))
214         if e.minor not in state:
215             return False, "step %d: minor premise %s not available" % (i, show(e.minor))
216         got = detach(e.major, e.minor, max_size)
217         if got is None:
218             return False, "step %d: D(%s,%s) is undefined" % (
219                 i, show(e.major), show(e.minor))
220         if got != e.concl:
221             return False, ("step %d: rule gives %s, proof claims %s"
222                            % (i, show(got), show(e.concl)))
223         state.add(got)
224     if target not in state:
225         return False, "final state does not contain the target"
226     return True, "%d steps, all valid detachments" % len(path)
227
228
229 # ============================================================================
```

```
#  commands
# ============================================================================
def cmd_sic(a):
    print("=" * 74)
    print("  E_{F,Y}  --  the canonical rule-enumeration SIC")
    print("=" * 74)
    print("\n  Gamma = %s" % ",  ".join(show(t) for t in GAMMA))
    print("  C(Gamma,1) = Gamma;  C(Gamma,m+1) = D_F(C(Gamma,m))\n")
    t0 = time.perf_counter()
    st = sic_stages(GAMMA, a.stages, a.max_size, cap=a.cap, say=print)
    print("\n  %d stages in %.1fs" % (len(st), time.perf_counter() - t0))
    print("\n  |C(Gamma,m)| = %s" % [len(s) for s in st])
    print("""
  This is v45's formal description of brute force: adjoin every direct
  consequence, no prioritising, no pruning.  tau is the index at which a
  target first appears in this sequence.

  Note it is NOT predator5.sweep(): that builds a graph whose nodes are sets
  of formulas, which is the right object for a search.  This has one state
  per stage.
""")
    for t, nm in NAMED.items():
        v = tau(st, t)
        print("    tau(%-9s) = %s        %s"
              % (nm, v if v else "> %d" % len(st), show(t)))
    print()


def cmd_compile(a):
    print("=" * 74)
    print("  thm:compilation  --  derived rules, and what they do to tau")
    print("=" * 74)

    ns = argparse.Namespace(depth=a.depth, max_size=a.max_size,
                            edge_cap=a.edge_cap, state_budget=20000)
    print("\n  harvesting targets...")
    _, _, targets = P5.sweep(GAMMA, a.depth, a.max_size, a.edge_cap, 20000)
    tr, te = split_targets(targets, a.test_frac, a.seed)
    print("    %d targets; %d compiled, %d held out" % (len(targets), len(tr), len(te)))

    print("\n  building E_{F,Y} without derived rules...")
    base = sic_stages(GAMMA, a.stages, a.max_size, cap=a.cap)
    print("    %d stages, |C| = %s" % (len(base), [len(s) for s in base]))

    D = compile_rules(GAMMA, [t for t, _ in tr])
    print("\n  compiling %d derived rules (one per training target)" % len(tr))
    aug = sic_stages(GAMMA, a.stages, a.max_size, rules=D, cap=a.cap)
    print("    %d stages, |C| = %s" % (len(aug), [len(s) for s in aug]))

    # --- the theorem's own claim, on the compiled targets -----------------
    bad = [t for t, _ in tr if (tau(aug, t) or 99) > 2]
    print("\n  THEOREM CHECK  (thm:compilation: tau <= 2 on compiled targets)")
    print("    %d of %d compiled targets reach tau <= 2   %s"
          % (len(tr) - len(bad), len(tr), "ok" if not bad else "<-- FAILED"))

    # --- the question the theorem does not answer -------------------------
    rows = []
    for t, d in te:
        t0, t1 = tau(base, t), tau(aug, t)
        rows.append((t, d, t0, t1))
    moved = [r for r in rows if r[2] and r[3] and r[3] < r[2]]
    same = [r for r in rows if r[2] and r[3] and r[3] == r[2]]
    unreached = [r for r in rows if not r[2]]

    print("\n  HELD-OUT TARGETS  (the cache was NOT built for these)")
    print("    %-9s %-8s %-8s %s" % ("geodesic", "tau_F", "tau_F^D", "formula"))
    print("    " + "-" * 62)
    for t, d, t0, t1 in sorted(rows, key=lambda r: (r[1], show(r[0])))[:a.show]:
        print("    %-9d %-8s %-8s %s"
              % (d, t0 if t0 else "-", t1 if t1 else "-", show(t)[:34]))
    if len(rows) > a.show:
        print("    ... %d more" % (len(rows) - a.show))

    print("\n    lowered   %d" % len(moved))
    print("    unchanged %d" % len(same))
    if unreached:
        print("    not reached by either  %d  (raise --stages or --cap)"
              % len(unreached))

    print("""
  READING THIS

    thm:compilation is a theorem about the COMPILED targets, and the check
    above is its direct test: each becomes available at stage 2 because its
    derived rule fires on Gamma.

    The held-out column is the part the theorem does not cover.  A cache built
    from one set of targets lowers tau for another only when the compiled
    formulas are useful PREMISES for the new goals -- that is, when the cache
    happens to contain lemmas rather than merely answers.

    If the held-out column shows little movement, the honest reading is that
```

```
    compiling answers is not the same as compiling lemmas, and that choosing
    WHICH formulas to compile is the real problem.  That choice is exactly
    where a ranker could earn its place -- and unlike reordering Sigma, it
    would be operating on the axis v45 actually formalises.
"""
    if a.out:
        json.dump(dict(compiled=len(tr), held_out=len(te),
                       lowered=len(moved), unchanged=len(same),
                       rows=[(show(t), d, t0, t1) for t, d, t0, t1 in rows]),
                  open(a.out, "w"), indent=2)
        print("  wrote %s\n" % a.out)


# ============================================================================
#  the materialisation count -- the second axis
# ============================================================================
def mu_sic(stages, target):
    """mu_M(Gamma, phi) = number of distinct FORMULAS the machine built before
    halting.

    tau counts stage advances; mu counts what those stages produced.  For the
    canonical rule-enumeration SIC each stage adjoins every direct consequence
    at once, so mu is |C(Gamma, tau)| -- the whole state at the halting stage.

    The two are independent.  The Proof-Horizon Theorem bounds tau below by
    ||phi||_F(Gamma) and nothing can beat it; NOTHING bounds mu, and that is
    exactly where a search policy operates.  Reordering Sigma leaves tau alone
    and lowers mu; derived rules lower tau and leave mu within a stage alone."""
    t = tau(stages, target)
    if t is None:
        return None
    return len(stages[t - 1])


def search_mu(gamma, target, policy, max_size, edge_cap, budget, lam=1.0):
    """Run a guided search, reporting (length, expansions, mu).

    mu here counts DISTINCT FORMULAS materialised -- the conclusions the search
    actually built -- so it is measured in the same units as mu_sic and the two
    can be compared.  Counting states instead would count sets of formulas and
    would not be comparable to |C(Gamma,m)|."""
    start = frozenset(gamma)
    if target in start:
        return 0, 0, 0
    seen = {start}
    built = set(gamma)
    frontier = [(0.0, 0, 0, start)]
    exp = tie = 0
    while frontier:
        _, depth, _, p = heapq.heappop(frontier)
        exp += 1
        if exp > budget:
            return None, exp, len(built)
        edges, sc = policy.order(p, admissible(p, max_size, edge_cap),
                                 gamma, target)
        for e, s in zip(edges, sc):
            qs = p | {e.concl}
            if qs in seen:
                continue
            seen.add(qs)
            built.add(e.concl)
            if e.concl == target:
                return depth + 1, exp, len(built)
            tie += 1
            heapq.heappush(frontier, (depth + 1 - lam * s, depth + 1, tie, qs))
    return None, exp, len(built)


def cmd_measure(a):
    """Both axes on the same targets: tau unchanged, mu moved."""
    print("=" * 74)
    print("  TAU AND MU  --  the two axes, measured together")
    print("=" * 74)

    print("\n  sweeping...")
    dist, pred, targets = P5.sweep(GAMMA, a.depth, a.max_size, a.edge_cap, 20000)
    tr, te = split_targets(targets, a.test_frac, a.seed)
    print("    %d targets, %d held out" % (len(targets), len(te)))

    print("  building the canonical SIC...")
    st = sic_stages(GAMMA, a.stages, a.max_size, cap=a.cap)
    print("    %d stages, |C| = %s" % (len(st), [len(x) for x in st]))

    pol = None
    if a.model != "none":
        print("  fitting a ranker on the training targets...")
        pairs = P5.build_pairs(GAMMA, dist, pred, tr, a.max_size, a.edge_cap,
                               2000, say=lambda *x: None)
        if pairs:
            r = P5.make_ranker(a.model, seed=a.seed, n_estimators=60,
                               max_depth=8, min_samples_leaf=4)
            r.fit(pairs, say=lambda *x: None)
```

```
            pol = P5.Policy(r, mode="reorder")
            print("    fitted on %s pairs" % f"{len(pairs):,}")

    print("\n  %-9s %-7s %-9s %-9s %-9s %s"
          % ("geodesic", "tau_E", "mu_E", "mu_BFS", "mu_pol", "formula"))
    print("  " + "-" * 68)
    rows = []
    for t, d in sorted(te, key=lambda td: td[1])[:a.show]:
        te_ = tau(st, t)
        me_ = mu_sic(st, t)
        _, _, mb = search_mu(GAMMA, t, P5.Policy(None), a.max_size,
                             a.edge_cap, a.budget, 0.0)
        mp = None
        if pol is not None:
            _, _, mp = search_mu(GAMMA, t, pol, a.max_size, a.edge_cap,
                                 a.budget, a.lam)
        rows.append((d, te_, me_, mb, mp))
        print("  %-9d %-7s %-9s %-9s %-9s %s"
              % (d, te_ or "-", me_ or "-", mb, mp if mp is not None else "-",
                 show(t)[:26]))

    ok = [r for r in rows if r[3] and r[4]]
    if ok:
        gm = sum(r[3] for r in ok) / len(ok)
        gp = sum(r[4] for r in ok) / len(ok)
        print("\n  mean mu:  BFS %.1f   policy %.1f   ratio %.2fx"
              % (gm, gp, gm / max(gp, 1e-9)))
    print("""
  tau_E is the stage at which the canonical machine first holds the target.
  mu_E is everything that machine built by then.  mu_BFS and mu_pol are what
  the two searches built.

  THE POINT.  A policy cannot change tau -- the corollary to Branch-Covering
  forbids reaching phi before ||phi||_F(Gamma), and reordering Sigma leaves the
  proof at the same depth.  It changes mu, and mu is unbounded by any theorem
  in the framework.  Naming it gives the second axis somewhere to live.
""")
    if a.out:
        json.dump([dict(geodesic=d, tau=t_, mu_sic=m_, mu_bfs=mb, mu_pol=mp)
                   for d, t_, m_, mb, mp in rows],
                  open(a.out, "w"), indent=2)
        print("  wrote %s\n" % a.out)


def cmd_family(a):
    """A family (Gamma_n, phi_n) for thm:asympspeedup.

    The theorem needs p, q with tau_{F^D} <= p(n), every F-proof at least
    q(n) long, and q(n)/p(n) unbounded.  Nothing in this project has ever
    constructed such a family, so the theorem has never been instantiated.

    The construction here uses the shortest-path principle to get q exactly.
    A formula first appearing at BFS layer d has ||phi|| = d -- not an
    estimate, a computed value -- so taking phi_d to be any target of geodesic
    exactly d gives q(d) = d BY MEASUREMENT.

    D compiles every target of geodesic strictly less than d.  Those are
    lemmas, not answers: phi_d is not among them.  If tau_{F^D}(phi_d) stays
    bounded while d grows, q/p grows without bound and the hypotheses of
    thm:asympspeedup are met on this family."""
    print("=" * 74)
    print("  A FAMILY FOR thm:asympspeedup")
    print("=" * 74)

    print("\n  sweeping to depth %d..." % a.depth)
    dist, pred, targets = P5.sweep(GAMMA, a.depth, a.max_size, a.edge_cap,
                                   a.state_budget)
    by_d = defaultdict(list)
    for t, d in targets.items():
        by_d[d].append(t)
    print("    targets by geodesic: %s"
          % {d: len(v) for d, v in sorted(by_d.items())})

    print("\n  %-4s %-8s %-8s %-10s %-10s %s"
          % ("d", "q(d)", "p(d)", "q/p", "|D|", "phi_d"))
    print("  " + "-" * 66)
    rows = []
    for d in sorted(by_d):
        if d < 2 or not by_d[d]:
            continue
        phi = sorted(by_d[d], key=lambda t: (size(t), show(t)))[0]
        lemmas = [t for dd, ts in by_d.items() if dd < d for t in ts]
        D = compile_rules(GAMMA, lemmas)
        st = sic_stages(GAMMA, a.stages, a.max_size, rules=D, cap=a.cap)
        p_d = tau(st, phi)
        q_d = d
        rows.append((d, q_d, p_d, len(lemmas)))
        print("  %-4d %-8d %-8s %-10s %-10d %s"
              % (d, q_d, p_d if p_d else "-",
                 "%.2f" % (q_d / p_d) if p_d else "-",
                 len(lemmas), show(phi)[:24]))

```

```
    got = [r for r in rows if r[2]]
    print("""
  q(d) is the exact shortest-proof length, computed by breadth-first search,
  not estimated -- this is the shortest-path principle doing the work.
  p(d) is the transaction count in F^D, where D holds every SHORTER target.
""")
    if len(got) >= 3:
        ratios = [r[1] / r[2] for r in got]
        pmax = max(r[2] for r in got)
        # "non-decreasing" is not the test.  q/p converging to a constant is
        # non-decreasing and is exactly what a NON-separation looks like.  The
        # hypothesis needs p BOUNDED while q grows, so check p directly.
        ps = [r[2] for r in got]
        p_bounded = len(set(ps[-3:])) == 1 if len(ps) >= 3 else False
        p_tracks_d = all(p == r[0] for r, p in zip(got[-2:], ps[-2:]))
        rising = (ratios[-1] > ratios[0] + 1e-9) and p_bounded
        print("  q/p over d = %s:  %s"
              % ([r[0] for r in got], ["%.2f" % x for x in ratios]))
        print("  p(d) bounded by %d across the family" % pmax)
        if p_tracks_d:
            print("""
  p(d) = d on the last rows, so q/p converges to 1 and the third hypothesis
  FAILS on this family.  Compiling every shorter target does not keep the
  transaction count bounded: each new depth still needs one more stage, because
  the derived rules deliver their targets at stage 2 and phi_d is one detachment
  further out than phi_{d-1}.

  That is a real negative result about THIS D, not about the theorem.  A family
  exhibiting separation needs derived rules whose targets are PREMISES for
  arbitrarily deep goals -- a chain lemma, not a layer of answers.""")
        elif rising:
            print("""
  q/p is non-decreasing and p is bounded on the range measured, which is what
  thm:asympspeedup's third hypothesis requires -- for these d.  It is EVIDENCE
  of a separation, not a proof of one: the theorem quantifies over all n, and
  a sweep reaches finitely many.  Extending d is the only way to strengthen
  it, and the state count grows fast.""")
        else:
            print("""
  q/p is not rising on this range, so the third hypothesis is not met here.
  That is a real answer: compiling every shorter target is not enough to keep
  p bounded, and the family needs a different D.""")
    else:
        print("  too few reachable d; raise --depth or --stages\n")
    if a.out:
        json.dump([dict(d=d, q=q, p=p, n_lemmas=n) for d, q, p, n in rows],
                  open(a.out, "w"), indent=2)
        print("\n  wrote %s\n" % a.out)


def cmd_prove(a):
    print("=" * 74)
    print("  PROVE  --  find a proof, then check it independently")
    print("=" * 74)

    _, _, targets = P5.sweep(GAMMA, a.depth, a.max_size, a.edge_cap, 20000)
    named = {v: k for k, v in NAMED.items()}
    if a.target in named:
        tgt = named[a.target]
    else:
        cands = [t for t in targets if show(t) == a.target]
        if not cands:
            print("\n  target not found.  Try one of: %s"
                  % ", ".join(sorted(named)))
            print("  or a formula string as printed by harvest.\n")
            return 1
        tgt = cands[0]

    d = targets.get(tgt)
    print("\n  target    %s   %s" % (show(tgt), NAMED.get(tgt, "")))
    print("  geodesic  %s  (computed by BFS)" % d)

    pol = P5.Policy(None)          # unguided; the point here is the proof
    t0 = time.perf_counter()
    ln, exp, path = search_with_path(GAMMA, tgt, pol, a.max_size, a.edge_cap,
                                     a.budget, 0.0)
    dt = time.perf_counter() - t0

    if ln is None:
        print("\n  NOT FOUND in %s expansions\n" % f"{exp:,}")
        return 1

    print("\n  found     length %d, %s expansions, %.3fs" % (ln, f"{exp:,}", dt))
    print("\n  proof")
    state = set(GAMMA)
    for i, e in enumerate(path, 1):
        print("    %2d.  D(%s, %s)" % (i, show(e.major), show(e.minor)))
        print("         = %s" % show(e.concl))
        state.add(e.concl)

    ok, msg = check_proof(GAMMA, path, tgt, a.max_size)
    print("\n  INDEPENDENT CHECK: %s" % ("PASSED" if ok else "FAILED"))
```

```
    print("    %s" % msg)
    print("""
  The check replays each step through `detach` rather than trusting the Edge
  the search produced.  A search reporting success and a proof that checks are
  different claims, and only the second is evidence.
""")
    return 0 if ok else 1


def cmd_axes(a):
    print("=" * 74)
    print("  THE TWO SPEEDUP AXES")
    print("=" * 74)
    print("""
  v45 measures speedup as the transaction count

      tau_M(Gamma, phi) = min { m : phi in C(Gamma, m) }

  which counts stage advances.  Two independent things can be reduced, and
  the paper formalises one of them.

    axis              quantity           lowered by        v45
    ------------------------------------------------------------------
    stages            tau                derived rules     thm:compilation
    work per stage    materialisations   reordering Sigma  --

  Predator_5 operates entirely on the second.  It CANNOT lower tau: the proof
  it finds sits at the same depth, and the corollary to thm:branchcovering
  says no target-search SIC reaches phi before ||phi||_F(Gamma).  What it
  lowers is how many states get built on the way.

  Conversely, derived rules lower tau without touching materialisations
  within a stage -- D_F still fires on every available tuple.

  Measured, on the condensed-detachment fragment:

    reordering Sigma      2.14x fewer materialisations, 1.76x wall clock,
                          tau unchanged
    derived rules         tau -> 2 on compiled targets (thm:compilation),
                          materialisations unchanged
    discarding (prune)    neither; strictly worse than BFS on both

  The framework has a name and a theorem for the first axis.  The second is
  measured but unnamed, and a second transaction-like quantity -- a
  materialisation count alongside tau -- would give it a home.
""")


def main():
    ap = argparse.ArgumentParser(prog="tau", description=__doc__,
            formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd")

    def common(p):
        p.add_argument("--max-size", type=int, default=14)
        p.add_argument("--stages", type=int, default=6)
        p.add_argument("--cap", type=int, default=40000,
                       help="pairs tried per stage; truncates D_F, reported")

    s = sub.add_parser("sic"); common(s)

    c = sub.add_parser("compile"); common(c)
    c.add_argument("--depth", type=int, default=4)
    c.add_argument("--edge-cap", type=int, default=12)
    c.add_argument("--test-frac", type=float, default=0.3)
    c.add_argument("--seed", type=int, default=0)
    c.add_argument("--show", type=int, default=20)
    c.add_argument("--out", default=None)

    ms = sub.add_parser("measure"); common(ms)
    ms.add_argument("--depth", type=int, default=4)
    ms.add_argument("--edge-cap", type=int, default=12)
    ms.add_argument("--test-frac", type=float, default=0.3)
    ms.add_argument("--seed", type=int, default=0)
    ms.add_argument("--budget", type=int, default=4000)
    ms.add_argument("--lam", type=float, default=0.5)
    ms.add_argument("--model", default="logistic",
                    choices=["logistic", "forest", "none"])
    ms.add_argument("--show", type=int, default=15)
    ms.add_argument("--out", default=None)

    fm = sub.add_parser("family"); common(fm)
    fm.add_argument("--depth", type=int, default=6)
    fm.add_argument("--edge-cap", type=int, default=12)
    fm.add_argument("--state-budget", type=int, default=40000)
    fm.add_argument("--out", default=None)

    p = sub.add_parser("prove"); common(p)
    p.add_argument("--target", default="identity")
    p.add_argument("--depth", type=int, default=4)
    p.add_argument("--edge-cap", type=int, default=12)
    p.add_argument("--budget", type=int, default=4000)
```

```
690
691     sub.add_parser("axes")
692
693     a = ap.parse_args()
694     if getattr(a, "edge_cap", None) == 0:
695         a.edge_cap = None
696     if a.cmd == "sic":       cmd_sic(a)
697     elif a.cmd == "compile": cmd_compile(a)
698     elif a.cmd == "measure": cmd_measure(a)
699     elif a.cmd == "family":  cmd_family(a)
700     elif a.cmd == "prove":   sys.exit(cmd_prove(a))
701     elif a.cmd == "axes":    cmd_axes(a)
702     else:                    ap.print_help()
703
704
705 if __name__ == "__main__":
706     main()
```